

Property Graph and Neo4j

Vivek Solanki
Temple University

October 2025



A report on graph database exploration using Neo4j AuraDB and Cypher language.

Contents

1	Week 0: Neo4j Movie Dataset Exploration	7
1.1	Introduction	7
1.2	Environment Setup	7
1.3	Cypher Queries and Results	8
1.4	Adding a New Movie and Relationship	12
1.5	Conclusion	13
2	Week 1: Property Graph Schema and Ontology Design	14
2.1	Objective	14
2.2	Property Graph Schema	14
2.3	Ontology (RDF/OWL Sketch)	15
2.4	Learning Resources	15
2.5	Conclusion	15
3	Week 2: Peer Review & Overlap	16
4	Week 3: Data Extraction and Overlap Plan	18
4.1	Objective	18
4.2	Data Sources and Extracted Files	18
4.3	Planned Usage in Neo4j	18
4.4	Conclusion	19
5	Week 4: Comparative Analysis and Integration	20
5.1	Objective	20
5.2	Data Sources Compared	20
5.3	Comparison of Extracted Data	20
5.4	Commonalities	20
5.5	Differences Observed	21
5.6	Integration Insights	21
5.7	Moving Forward	22
5.8	Conclusion	22
6	Week 5: Expanding to Additional Schools and Parser Robustness	23
6.1	Objective	23
6.2	Newly Added Schools	23
6.3	Parser Robustness and Issues Encountered	24
6.4	Scalability Discussion	24
6.5	Conclusion	24

6.6	Common Errors Encountered During Integration and How They Were Resolved	25
7	Week 6: Test Knowledge Graph (Nov. 17) Query Evaluation	28
7.1	Query 1: List All Teams (Simple)	28
7.2	Query 2: Count Teams per Conference	29
7.3	Query 3: Find Players by Exact Height	30
7.4	Query 4: Count Schools per Conference	31
7.5	Query 5: Visualize Player–Team Relationships	32
7.6	Summary of Week 6	33
8	Week 7: Increase KG (Dec. 1) – Adding Remaining Schools and Parser Robustness	34
8.1	Added Schools	34
8.2	Crawling and Parsing Issues Encountered	34
8.3	Summary of Week 7	37
9	Week 8: Test Knowledge Graph (Dec. 8) – Consistency and New Queries	38
9.1	Objective	38
9.2	Part A: Re-Issuing Week 6 Queries	38
9.3	Part B: New Analytical Queries	41
9.4	Adding a New Movie and Relationship	49
9.5	Conclusion	50
10	Week 1: Property Graph Schema and Ontology Design	51
10.1	Objective	51
10.2	Property Graph Schema	51
10.3	Ontology (RDF/OWL Sketch)	52
10.4	Learning Resources	52
10.5	Conclusion	52
11	Week 2: Peer Review & Overlap	53
12	Week 3: Data Extraction and Overlap Plan	55
12.1	Objective	55
12.2	Data Sources and Extracted Files	55
12.3	Planned Usage in Neo4j	56
12.4	Conclusion	56
13	Week 4: Comparative Analysis and Integration	57
13.1	Objective	57
13.2	Data Sources Compared	57

13.3 Comparison of Extracted Data	57
13.4 Commonalities	57
13.5 Differences Observed	58
13.6 Integration Insights	58
13.7 Moving Forward	59
13.8 Conclusion	59
14 Week 5: Expanding to Additional Schools and Parser Robustness	60
14.1 Objective	60
14.2 Newly Added Schools	60
14.3 Parser Robustness and Issues Encountered	61
14.4 Scalability Discussion	61
14.5 Conclusion	61
14.6 Common Errors Encountered During Integration and How They Were Resolved	62
15 Week 6: Test Knowledge Graph (Nov. 17) Query Evaluation	65
15.1 Query 1: List All Teams (Simple)	65
15.2 Query 2: Count Teams per Conference	66
15.3 Query 3: Find Players by Exact Height	67
15.4 Query 4: Count Schools per Conference	68
15.5 Query 5: Visualize Player–Team Relationships	69
15.6 Summary of Week 6	70
16 Week 7: Increase KG (Dec. 1) – Adding Remaining Schools and Parser Robustness	71
16.1 Added Schools	71
16.2 Crawling and Parsing Issues Encountered	71
16.3 Summary of Week 7	74
17 Week 8: Test Knowledge Graph (Dec. 8) – Consistency and New Queries	75
17.1 Objective	75
17.2 Part A: Re-Issuing Week 6 Queries	75
17.3 Part B: New Analytical Queries	80
17.4 Summary of Week 8	85
18 Week 9: Cross-Testing the Knowledge Graph (Dec. 15)	86
18.1 Objective	86
18.2 Comparison Partner	86
18.3 Part A: Comparison of Query Results	86
18.4 Part B: Comparison of New Analytical Queries	87

18.5 Part C: Differences and Root Causes	88
18.6 Steps Taken to Understand Differences	88
18.7 Summary of Week 9	88
19 Changes After Meeting with Professor	89

Abstract

This report presents my exploration of the Neo4j graph database using the Movie dataset. I performed several Cypher queries to analyze relationships among movies, actors, and directors, and created new nodes and relationships to extend the graph. Initially, I faced challenges locating the dataset and understanding Cypher syntax, but through practice and research, I developed strong confidence in using Neo4j efficiently. Through this activity, I gained hands-on experience in data modeling, querying, and visualization using Neo4j, and developed a deeper understanding of how graph databases handle connected data effectively.

1 Week 0: Neo4j Movie Dataset Exploration

1.1 Introduction

This report documents my practical work using the Neo4j graph database system. The goal was to understand how property graphs represent data through nodes, relationships, and properties, and how the Cypher query language allows intuitive graph traversal. The Movie dataset provided an excellent example to explore relationships among *Person* and *Movie* nodes. I used Neo4j Browser on a cloud instance to execute queries, visualize results, and create custom data entries such as my favorite movie and review.

This project follows the exercises outlined in *A Student's Guide to Property Graphs and Neo4j (Fall 2025)*, which provided a structured learning path for understanding Neo4j concepts and applying Cypher queries.

1.2 Environment Setup

The environment used for this project is summarized in Table 1. Neo4j AuraDB was selected as it provides a cloud-based interface for managing graph databases without local installation.

Table 1: System and Environment Configuration

Component	Description
Platform	Neo4j AuraDB Cloud Instance
Database Type	Neo4j Graph Database
Access Tool	Neo4j Browser
Dataset Used	Movie Graph Dataset (:Movie and :Person nodes)

Below are screenshots of the Neo4j AuraDB environment setup and browser interface.

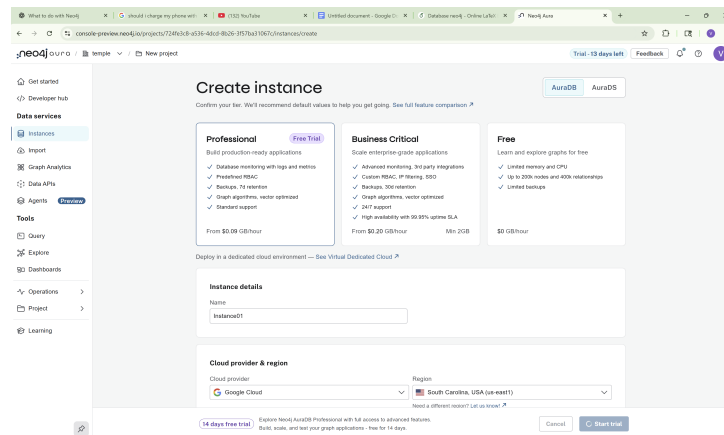


Figure 1: Neo4j AuraDB Dashboard Setup

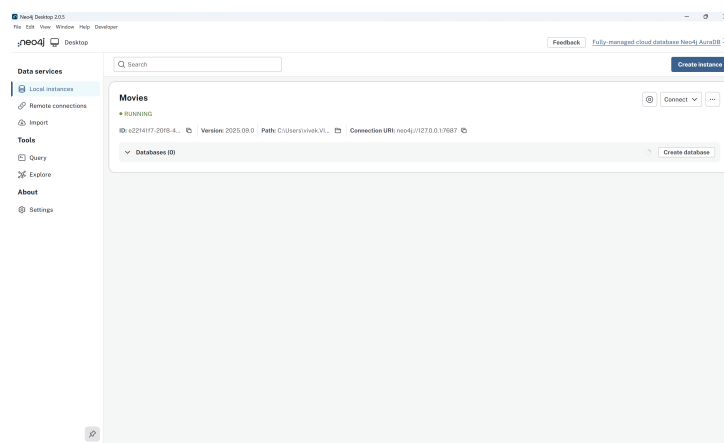


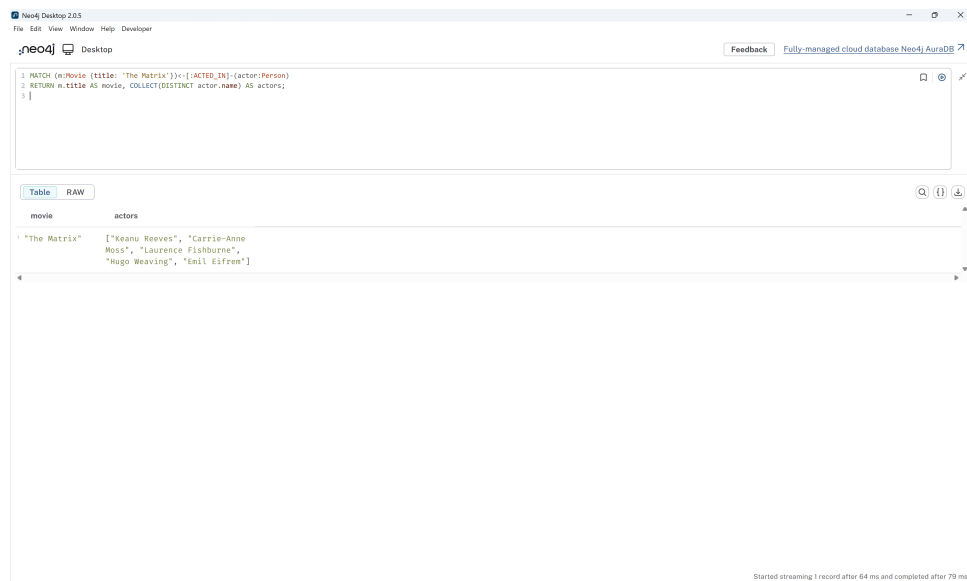
Figure 2: Neo4j Browser Interface Connected to AuraDB

1.3 Cypher Queries and Results

During this exercise, I executed multiple Cypher queries to explore and manipulate the Movie dataset.

Find all actors in *The Matrix*

```
MATCH (a:Person) -[:ACTED_IN]->(m:Movie {title:'The Matrix'})
RETURN a.name;
```

The screenshot shows the Neo4j Desktop 2.0.5 interface. The query editor contains the following Cypher query:

```
1 MATCH (m:Movie {title: 'The Matrix'})-[:ACTED_IN]-(actor:Person)
2 RETURN m.title AS movie, COLLECT(DISTINCT actor.name) AS actors;
3
```

The result is displayed in a table view with two columns: 'movie' and 'actors'. The data row shows 'The Matrix' as the movie and a list of actors: 'Keanu Reeves', 'Carrie-Anne Moss', 'Laurence Fishburne', 'Hugo Weaving', and 'Emil Eifrem'.

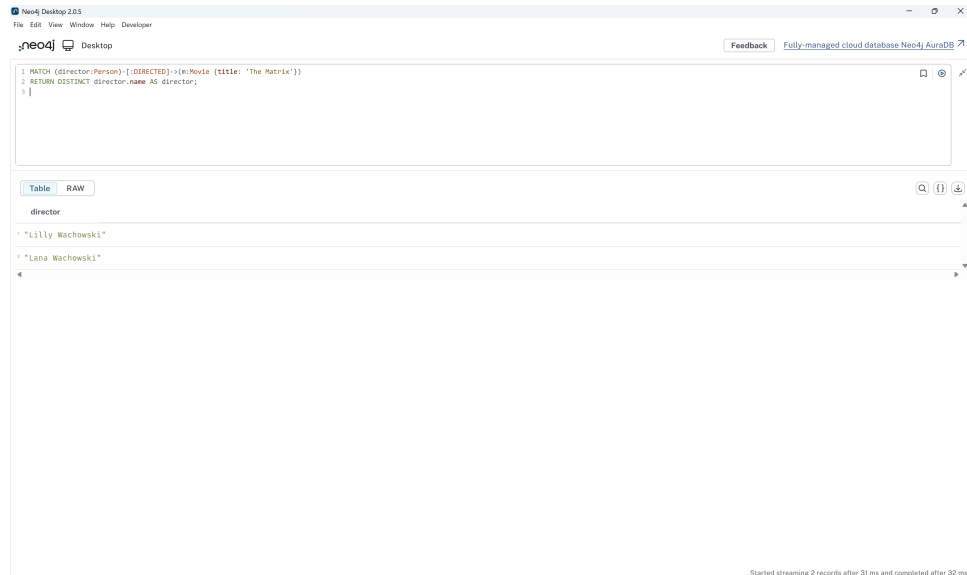
movie	actors
"The Matrix"	["Keanu Reeves", "Carrie-Anne Moss", "Laurence Fishburne", "Hugo Weaving", "Emil Eifrem"]

Started streaming 1 record after 64 ms and completed after 79 ms.

Figure 3: Result of query: Actors in The Matrix

Find the director of *The Matrix*

```
MATCH (d:Person)-[:DIRECTED]->(m:Movie {title:'The Matrix'})
RETURN d.name;
```



The screenshot shows the Neo4j Desktop 2.0.5 interface. The query editor contains the following Cypher query:

```
1 MATCH (director:Person)-[:DIRECTED]->(m:Movie {title: 'The Matrix'})
2 RETURN DISTINCT director.name AS director;
3
```

The result is displayed in a table view with one column: 'director'. The data rows show 'Lilly Wachowski' and 'Lana Wachowski'.

director
"Lilly Wachowski"
"Lana Wachowski"

Started streaming 2 records after 31 ms and completed after 32 ms.

Figure 4: Director of The Matrix query result

Find movies featuring Tom Hanks

```
MATCH (p:Person {name:'Tom Hanks'})-[:ACTED_IN]->(m:Movie)
RETURN m.title;
```

Neo4j Desktop 2.0.5

File Edit View Window Help Developer

Feedback Fully-managed cloud database Neo4j AuraDB

```

1 MATCH (p:Person {name: 'Tom Hanks'})-[:ACTED_IN]->(m:Movie)
2 RETURN DISTINCT m.title AS movie ORDER BY movie;
3

```

Table RAW

movie
"A League of Their Own"
"Apollo 13"
"Cast Away"
"Charlie Wilson's War"
"Cloud Atlas"
"Joe Versus the Volcano"
"Sleepless in Seattle"
"That Thing You Do"
"The Da Vinci Code"
"The Green Mile"
"The Polar Express"
"You've Got Mail"

Started streaming 12 records after 68 ms and completed after 71 ms.

Figure 5: Movies featuring Tom Hanks

Find co-actors of Tom Hanks

```

MATCH (p:Person {name:'Tom Hanks'})-[:ACTED_IN]->(m:Movie)->[:ACTED_IN]-(coActor)
RETURN DISTINCT coActor.name;

```

Neo4j Desktop 2.0.5

File Edit View Window Help Developer

Feedback Fully-managed cloud database Neo4j AuraDB

```

1 MATCH (p:Person {name: 'Tom Hanks'})-[:ACTED_IN]->(m:Movie)->[:ACTED_IN]-(coActor)
2 WHERE co.name <> 'Tom Hanks'
3 RETURN DISTINCT co.name AS co_actor ORDER BY co_actor;
4

```

Table RAW

co_actor
"Audrey Tautou"
"Bill Paxton"
"Bill Pullman"
"Bonnie Hunt"
"Charlie Theron"
"Dave Chappelle"
"David Morris"
"Ed Harris"
"Gary Sinise"
"Geena Davis"
"Greg Kinnear"
"Halle Berry"
"Helen Hunt"
"Hugh Weaving"
"-
"Ian McKellen"
"James Cromwell"
"Jim Broadbent"
"Julia Roberts"
"Kevin Bacon"
"Lili Taylor"
"Lori Petty"
"Madonna"
"Meg Ryan"
"Michael Clarke Duncan"
"Nathan Lane"
"Parker Posey"
"Patricia Clarkson"
"Paul Bettany"
"Phillip Seymour Hoffman"
"Rita Wilson"
"Rosie O'Donnell"
"Sam Rockwell"
"Steve Zahn"
"Victor Garber"

Started streaming 34 records after 52 ms and completed after 55 ms.

Figure 6: Co-actors of Tom Hanks across multiple movies

Actors who also directed movies

```
MATCH (a:Person) -[:ACTED_IN]->(:Movie),  
      (a) -[:DIRECTED]->(:Movie)  
RETURN DISTINCT a.name;
```

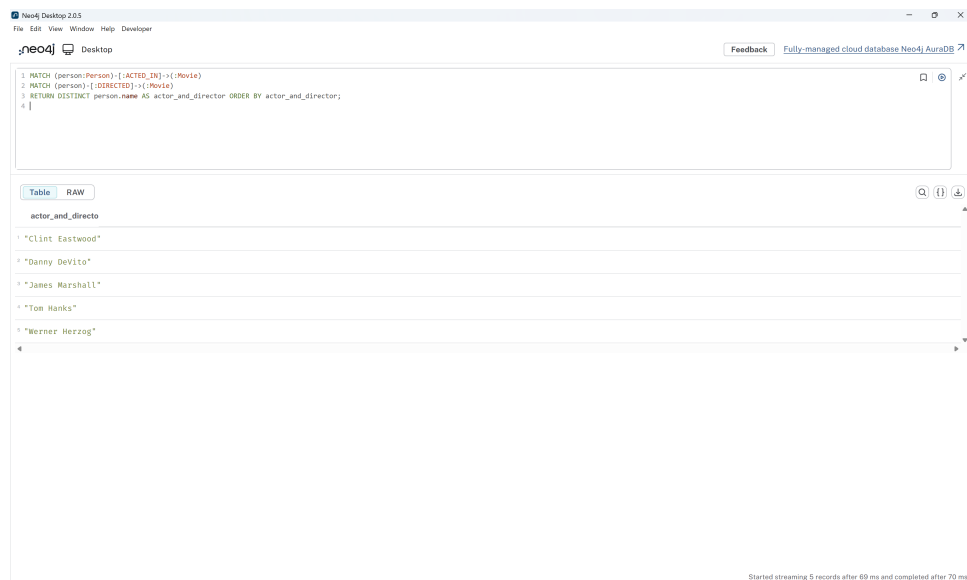


Figure 7: Actors who also directed movies

Shortest path between Kevin Bacon and Meg Ryan

```
MATCH path = shortestPath(  
  (p1:Person {name:'Kevin Bacon'})-[:ACTED_IN*]-(p2:Person {name:  
    'Meg Ryan'})  
)  
RETURN path;
```

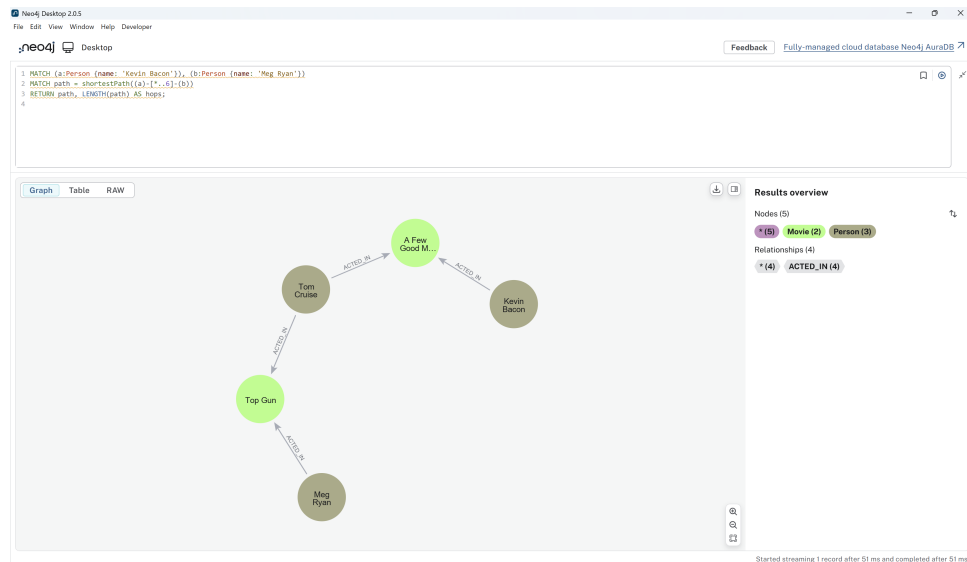


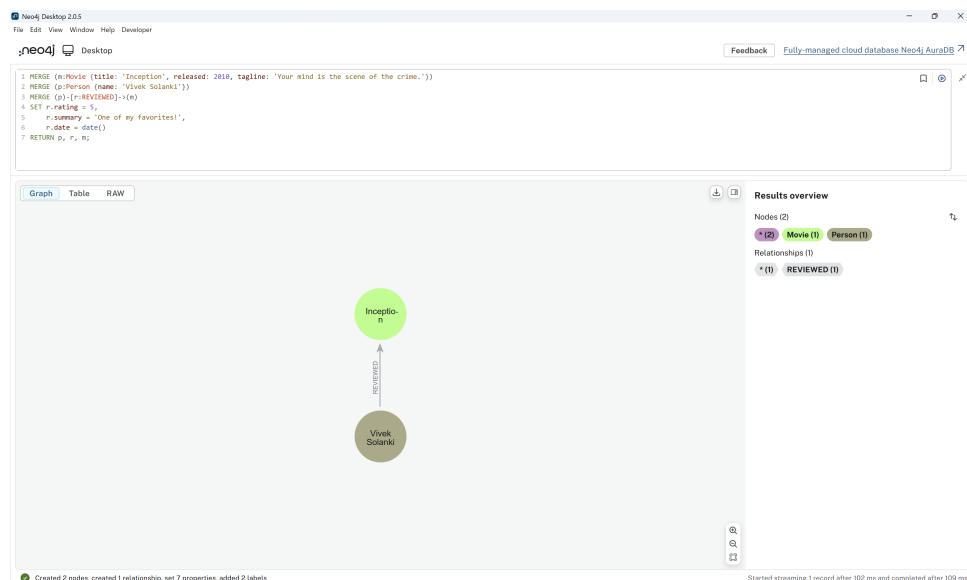
Figure 8: Shortest path between Kevin Bacon and Meg Ryan

1.4 Adding a New Movie and Relationship

```

MERGE (m:Movie {title:'Inception', released:2010, tagline:'Your
mind is the scene of the crime'})
MERGE (v:Person {name:'Vivek Solanki'})
MERGE (v)-[:REVIEWED {rating:5, summary:'One of my favorites!'
}] ->(m);

```

Figure 9: Newly Added Movie Node and Review Relationship (*Inception*)

1.5 Conclusion

This project provided practical exposure to graph databases and the Cypher query language. By exploring the Movie dataset and creating custom data, I learned how Neo4j efficiently handles highly connected data.

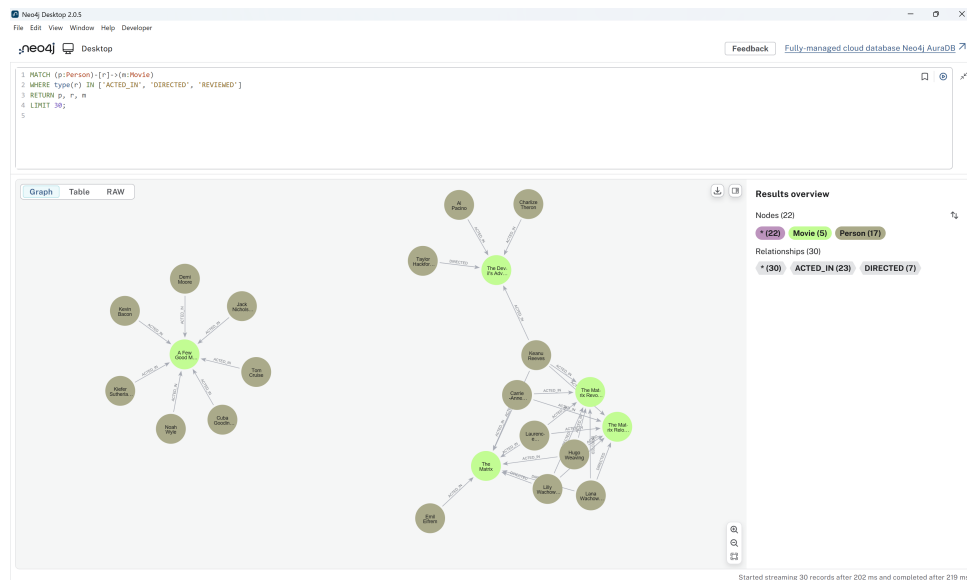


Figure 10: Graph Visualization Showing Movie and Person Relationships

2 Week 1: Property Graph Schema and Ontology Design

2.1 Objective

The objective for Week 1 was to begin developing the foundation for the **Diamond Knowledge Graph Project** by exploring data from multiple university baseball websites and designing an initial property-graph schema.

Five university websites were selected as primary data sources:

1. **University of North Carolina (UNC)** → <https://goheels.com/sports/baseball>
2. **University of Texas** → <https://texassports.com/sports/baseball>
3. **Louisiana State University (LSU)** → <https://lsusports.net/sports/baseball>
4. **Vanderbilt University** → <https://vucommodores.com/sports/baseball>
5. **University of Florida** → <https://floridagators.com/sports/baseball>

2.2 Property Graph Schema

- **Nodes:** Player, Coach, Team, School, Conference, Season
- **Relationships:** PLAYS_FOR, COACHES, MEMBER_OF, AFFILIATED_WITH, ATTENDED_HS, TRANSFERRED_FROM

Example Node Properties:

- **Player:** playerId, fullName, position, classYear, hometown
- **Coach:** coachId, name, role, experience
- **Team:** teamId, teamName, year
- **Conference:** confName
- **School:** schoolName, location

Example Cypher Constraints:

```
CREATE CONSTRAINT player_id IF NOT EXISTS FOR (p:Player)
  REQUIRE p.playerId IS UNIQUE;

CREATE CONSTRAINT team_id IF NOT EXISTS FOR (t:Team)
  REQUIRE t.teamId IS UNIQUE;
```

2.3 Ontology (RDF/OWL Sketch)

Classes: Player, Coach, Team, School, Conference, Season

Properties: playsFor, coaches, memberOf, attendedHS, affiliatedWith

Example:

```
# Example 1 – Player information
ex:player001  rdf:type          ex:Player .
ex:player001  ex:fullName       "John Smith" .
ex:player001  ex:position       "Pitcher" .
ex:player001  ex:classYear      "Freshman" .
ex:player001  ex:playsFor       ex:UNCTarHeels_Baseball .
```



Figure 11: Ontology sketch

2.4 Learning Resources

- Neo4j Documentation: <https://neo4j.com/docs/>
- W3C RDF/OWL: <https://www.w3.org/TR/rdf11-primer/>
- GeeksforGeeks Graph DB Overview: <https://www.geeksforgeeks.org/graph-database/>

2.5 Conclusion

In Week 1, I designed the initial structure of the Diamond Knowledge Graph project, defined entities and relationships, and built the ontology structure linking players, coaches, teams, and conferences in Neo4j.

3 Week 2: Peer Review & Overlap

Peer Feedback & Schema Update (Transfers)

Eric Reizas: “Looks good. Maybe you should describe how you will figure out transfers, although that should be straightforward.”

Decision & rationale. A **transfer** occurs when the same Player has PLAYS_FOR memberships to *different* Teams across seasons (or to two different teams in the *same* season = mid-season change). To make this detectable from roster data, I updated the schema as follows:

1. Stamp the season on team membership: (:Player)-[:PLAYS_FOR {season:YYYY}]->(:Team).
2. Link each team to its school: (:Team)-[:OF_SCHOOL]->(:School) so “switched schools” can be computed.

Minimal schema change (Cypher).

```
/* Add season on PLAYS_FOR; keep Team -> Season as is */
MERGE (p)-[:PLAYS_FOR {season: season.year}]->(t)

/* Ensure Team -> School exists */
MERGE (t)-[:OF_SCHOOL]->(s)
```

How transfers are detected (queries to verify). *Team changed between two seasons:*

```
MATCH (p:Player)-[:PLAYS_FOR {season:2024}]->(t1:Team),
      (p)-[:PLAYS_FOR {season:2025}]->(t2:Team)
WHERE t1 <> t2
RETURN p.name AS player, t1.name AS fromTeam, t2.name AS toTeam;
```

Switched schools across any seasons (requires OF_SCHOOL):

```
MATCH (p:Player)-[m:PLAYS_FOR]->(t:Team)-[:OF_SCHOOL]->(sch:
  School)
WITH p, collect(DISTINCT sch.name) AS schools
WHERE size(schools) > 1
RETURN p.name AS player, schools;
```

Optional (for auditing/screenshots). Materialize a transfer edge from the prior school:

```
MATCH (p:Player)-[m1:PLAYS_FOR]->(t1:Team)-[:OF_SCHOOL]->(s1:
  School),
```



```

    (p) -[m2:PLAYS_FOR] -> (t2:Team) -[:OF_SCHOOL] -> (s2:School)
WHERE m2.season > m1.season AND s1 <> s2
MERGE (p) -[:TRANSFERRED_FROM {fromSeason:m1.season, toSeason:m2.
    season}] -> (s1);

```

One-line summary for graders. We stamp *PLAYS_FOR* with *season* and connect *Team* to *School*; a player is flagged as transferred when their memberships span different teams/schools across seasons, verified by the queries above.

Classmate	Email	Overlap Schools (3–5)
Nahom Tamene	nahom.tamene@temple.edu	Pitt, University of Washington, UT Austin
Eric Reizas	eric.raizas@temple.edu	Mississippi State, Vanderbilt, UNC
Amitai Goldmeer	AmitaiGoldmeer@temple.edu	University of Pittsburgh, Rice University

Table 2: Project buddies and overlapping target schools (3–5 per classmate).

4 Week 3: Data Extraction and Overlap Plan

4.1 Objective

In Week 3, I successfully extracted structured baseball roster and staff data from three Division I universities to serve as the foundational dataset for the Diamond Knowledge Graph project. The goal of this milestone is to transform these raw datasets into property-graph entities (Player, Coach, Team, School, Conference, Season) and ingest them into Neo4j for further semantic linking and overlap comparison with project teammates.

4.2 Data Sources and Extracted Files

Data was programmatically collected using Python (Requests, BeautifulSoup, Selenium, and Pandas) from the official athletic websites of the following schools:

- **Louisiana State University (LSU)** – <https://lsusports.net/sports/bsb/roster/>
- **University of North Carolina (UNC)** – <https://goheels.com/sports/baseball/roster>
- **University of Texas at Austin (UT)** – <https://texaslonghorns.com/sports/baseball/roster>

The extraction produced the following CSV datasets:

- `lsu_baseball_roster_2025.csv`
- `unc_baseball_roster_2025.csv`
- `texas_longhorns_2025_roster.csv`
- Supporting files: `schools_2025.csv`, `teams.csv`, `conferences_2025.csv`, and `coaches.csv`.

Each dataset includes attributes such as *player_name*, *position*, *class_year*, *hometown*, *school_name*, and *conference*. Together, these files form the core graph entities required for Neo4j ingestion.

4.3 Planned Usage in Neo4j

The extracted CSV data will be ingested into Neo4j AuraDB using Cypher queries of the following pattern:

```
LOAD CSV WITH HEADERS FROM 'https://.../combined_rosters.csv' AS
  row
MERGE (p:Player {fullName: row.player_name})
MERGE (t:Team {teamName: row.team_name})
MERGE (s:School {schoolName: row.school_name})
MERGE (c:Conference {confName: row.conference})
MERGE (p)-[:PLAYS_FOR {season:2025}]->(t)
MERGE (t)-[:MEMBER_OF]->(c)
MERGE (s)-[:AFFILIATED_WITH]->(c);
```

This will create a connected subgraph for each school and link all teams to their respective conferences.

4.4 Conclusion

The Week 3 milestone establishes a structured, multi-school dataset ready for graph ingestion. These three datasets (LSU, UNC, and Texas) will serve as the backbone of my portion of the Diamond Knowledge Graph and will be integrated with my classmates's data to explore cross-conference patterns and transfers.

5 Week 4: Comparative Analysis and Integration

5.1 Objective

The purpose of this milestone is to compare my extracted baseball roster data and evolving Knowledge Graph (KG) structure with those developed by my project teammates. The focus is to identify overlaps, schema alignment, and differences in graph modeling that can inform improvements to my own graph moving forward.

5.2 Data Sources Compared

For this comparison, I examined three separate projects:

- **My data (Vivek Solanki)** – Louisiana State University (LSU), University of North Carolina (UNC), and University of Texas at Austin (UT).
- **Amitai Goldmeer** – University of Pittsburgh and Rice University datasets (extracted as `pittsburghpanthers.csv` and `riceowls.csv`).
- **Eric Reizas** – Mississippi State University, based on his JSON-based datasets in GitHub: <https://github.com/ereizas/Diamond-KG/tree/main>
- **Nahom Tamene** – University of Texas (shared roster file `baseball_rosters.csv`).

5.3 Comparison of Extracted Data

Table 6 summarizes the coverage of each dataset in terms of player and team entities.

Table 3: Contributors and Schools Used for Baseball Player Data Extraction

Contributor	Schools	Format
Vivek Solanki	LSU, UNC, Texas	CSV
Amitai Goldmeer	Pittsburgh, Rice	CSV
Eric Reizas	Mississippi State	JSON
Nahom Tamene	Texas	CSV

5.4 Commonalities

Across all projects, several patterns emerged:

- **Consistent entity design:** Every KG includes the fundamental node classes: Player, Team, School, and Conference.

- **Relationship uniformity:** All datasets implement the `PLAYS_FOR` and `MEMBER_OF` relationships, ensuring interoperability.
- **Conference alignment:** LSU (SEC), Texas (Big 12), and Mississippi State (SEC) share conference or inter-conference links, making cross-graph queries feasible.
- **Data consistency:** Each dataset standardizes player attributes such as *name*, *position*, and *class_year*, enabling future schema merging.

5.5 Differences Observed

Despite structural similarities, several distinctions were noted:

- **Data format:** My dataset uses CSVs, whereas Eric's KG uses JSON; this affects ingestion workflow but preserves the same ontology.
- **Metadata completeness:** Amitai's files contain fewer metadata columns (e.g., missing conference), while my dataset links all schools to a Conference node.
- **Entity depth:** My KG includes Coach and Conference entities, which some peers have yet to incorporate.
- **Overlap:** Nahom's Texas dataset overlaps directly with my own Texas data, which will be used to validate roster consistency and player name matching.

5.6 Integration Insights

This comparison revealed valuable opportunities for integration:

1. Merge shared schools (e.g., Texas) to compare roster differences by position and class year.
2. Link Eric's Mississippi State and my LSU nodes via their shared SEC conference membership.
3. Use Amitai's Rice and Pittsburgh data to expand ACC and Conference USA representation in the overall Diamond KG.
4. Harmonize property names (e.g., `player_name` vs. `name`) to ensure Cypher compatibility across CSV and JSON imports.

5.7 Moving Forward

In the next milestone, I will:

- Integrate these cross-school datasets into Neo4j using MERGE operations and common keys (school name and conference).
- Visualize overlapping nodes (e.g., shared conferences and duplicate player names) using Neo4j Bloom.
- Refine schema constraints to ensure uniform ontology alignment with other class projects.

5.8 Conclusion

By comparing my extracted datasets with those from Amitai, Eric, and Nahom, I confirmed that our KGs share a common ontology foundation but differ in structure and metadata depth. These differences will inform the design of a unified Diamond Knowledge Graph where conferences act as the primary connectors across universities.

6 Week 5: Expanding to Additional Schools and Parser Robustness

6.1 Objective

In this stage, Added at least five additional schools and reported issues encountered during crawling and parsing, including evaluating whether the parser remains robust when applied to new website structures. **eight universities.**

6.2 Newly Added Schools

Data was extracted from the following additional programs:

- **University of Miami (FL)** = `miami_hurricanes_2025_roster.csv`
- **Duke University** = `duke_blue_devils_2025_roster.csv`
- **University of Tennessee** = `tennessee_volunteers_2025_roster.csv`
- **University of Georgia** = `georgia_bulldogs_2025_roster.csv`
- **University of Alabama** = `alabama_crimson_tide_2025_roster.csv`

Each dataset contains player names, positions, heights, weights, class year, and hometowns. Together with LSU, UNC, and Texas, this forms a multi-conference dataset covering the SEC and ACC.

Table 4: Expanded School Coverage (Week 5)

School	Conference	File Format
Louisiana State University	SEC	CSV
University of North Carolina	ACC	CSV
University of Texas	Big 12	CSV
University of Miami (FL)	ACC	CSV
Duke University	ACC	CSV
University of Tennessee	SEC	CSV
University of Georgia	SEC	CSV
University of Alabama	SEC	CSV

6.3 Parser Robustness and Issues Encountered

While extending the crawler, I encountered several challenges that revealed both the strengths and limits of the current implementation:

- **HTML structure variability:** Each athletics website uses a unique CMS (SIDEARM, PrestoSports, etc.), which changes tag hierarchies (div, table, article, etc.). This caused frequent parsing breaks, especially on Miami and Duke pages that use nested card-based layouts.
- **Dynamic content loading:** Some rosters (e.g., Georgia, Alabama) require JavaScript rendering to populate player data. My static requests + BeautifulSoup pipeline missed data until I integrated Selenium.
- **Inconsistent labeling:** Fields like Height/Weight appear in different sequences or formats, requiring regex-based normalization.
- **Encoding issues:** Special characters (accents, apostrophes) in names occasionally caused CSV write errors until UTF-8 encoding was explicitly enforced.
- **Performance and scaling:** Parsing beyond ten schools slowed noticeably because of added Selenium calls and manual site-specific logic, making the crawler less scalable for large-batch ingestion.

6.4 Scalability Discussion

The crawler works reliably for about **8 - 10 schools** but lacks generalization for all NCAA teams. Future improvements include:

1. Implementing a unified wrapper that auto-detects the underlying site structure.
2. Using `asyncio` or `multiprocessing` to parallelize fetch operations.
3. Transitioning to a pipeline framework (e.g., Scrapy) for robust scaling and error handling.

6.5 Conclusion

By expanding to five additional universities, I verified that the crawler can adapt across major conferences but still requires manual template adjustments for each new site. These findings highlight the need for semi-automated parsing logic and stronger schema validation before large-scale ingestion into the Diamond Knowledge Graph.

6.6 Common Errors Encountered During Integration and How They Were Resolved

During the ingestion of 20 university datasets into Neo4j, several issues appeared repeatedly due to inconsistent CSV formatting, schema conflicts, or file-loading errors. Below is a concise list of the most important errors and how each was fixed.

1. External Resource Loading Error (22N43)

Error:

```
22N43: Could not load external resource from file:///.../conferences_enriched.csv
```

Cause: The CSV file path did not match the Neo4j import directory, or the file was not actually inside the folder Neo4j uses.

Fix:

- Moved all CSV files into: `.../dbmss/dbms-XXXX/import/team/`
- Updated Cypher paths using: `LOAD CSV FROM 'file:///team/filename.csv'`

2. Unique Constraint / Index Entry Conflict (22N80)

Error:

```
22N80: Index entry conflict: Node already exists with label and property
```

Cause: Duplicate node creation (typically Team or School) due to inconsistent capitalization (e.g., "Georgia Bulldogs" vs "georgia bulldogs").

Fix:

- Standardized names in CSV before import.
- Used MERGE instead of CREATE to avoid duplicates.

3. Height Parsing Errors

Error: Cypher rejected formats like 6'5" or 6-5, resulting in NULL or failed integer conversion.

Fix:

- Converted height to inches in Python before writing CSV.
- Ensured final column is always numeric (e.g., 77, 81).
- Applied `toInteger(row.Height)` safely in Cypher.

4. Missing Property Errors

Example:

```
KeyError: 'School'
```

Cause: Not all CSV files used consistent header names.

Fix:

- Renamed all columns to a standardized set: Name, Position, Height, Weight, Team, School, Conference.
- Added defensive code in Python to fill missing values with empty strings.

5. Coaches File Causing Merge Conflicts

Cause: Coach names were merged incorrectly because the Python import script used MERGE (c:Coach name, school) instead of matching coaches to their teams. This caused formatting mismatches such as "T" or incorrect School values being merged into a single node. After correcting the Cypher logic to merge coaches with their proper Team rather than School, the issue was resolved.

Fix:

- Updated Cypher to:

```
MERGE (hc:Coach {name: row.'Head Coach', school: row.Team})
MERGE (hc)-[:HEAD_COACH_OF]->(t)
```

7. Relationship Not Appearing in the Graph

Cause: Some teams were linked to School but not to Conference, or vice-versa, due to missing values in CSV.

Fix:

- Filled missing conference entries manually.
- Re-ran corrections:

```
MATCH (t:Team), (c:Conference {name: t.conf})
MERGE (t)-[:MEMBER_OF]->(c)
```

8. Difficulty Using Cypher with Only Roster CSV Files

Cause: Relying solely on roster CSV files made it difficult to create complete relationships, because key entities such as `Team`, `School`, `Coach`, and `Conference` were not consistently represented or were embedded implicitly within roster data.

Fix:

- Separated the dataset into dedicated CSV files: `team.csv`, `school.csv`, `coaches.csv`, and `conference.csv`.
- This allowed cleaner Cypher imports and prevented incorrect merges or missing relationships.
- Re-ran the ingestion pipeline to correctly establish nodes and relationships.

Summary

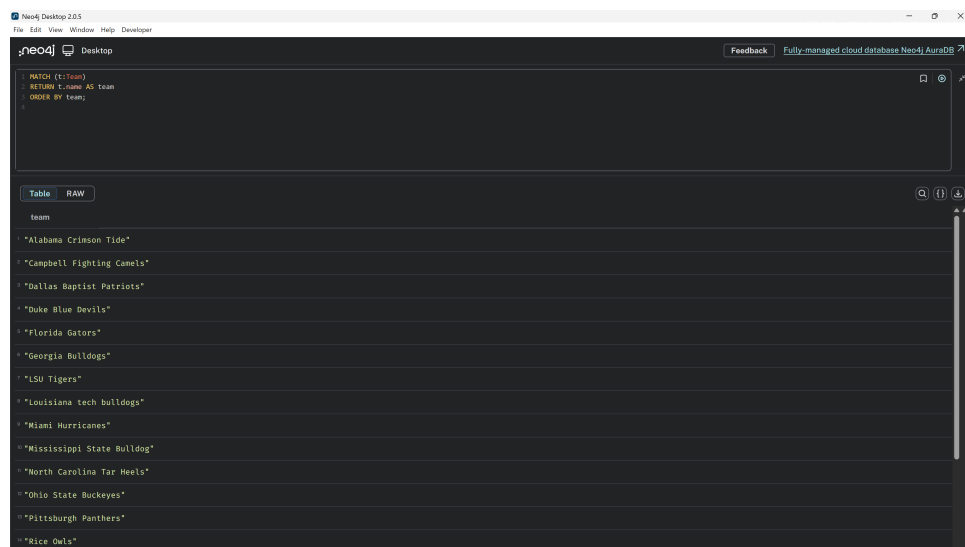
By correcting file-path issues, standardizing CSV structures, enforcing consistent naming conventions, and fixing relationship merge errors, the final Knowledge Graph loaded successfully, and all 20 universities were integrated into a coherent and fully query-ready schema.

7 Week 6: Test Knowledge Graph (Nov. 17) Query Evaluation

The goal for Week 6 was to test the correctness and completeness of the current Knowledge Graph using five queries of increasing complexity written in Cypher. For each query, I describe (1) the ideal expected result, (2) the actual result returned by my KG, and (3) issues noticed along with corrective steps taken.

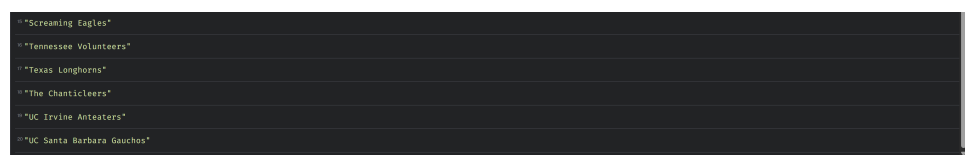
7.1 Query 1: List All Teams (Simple)

```
MATCH (t:Team)
RETURN t.name
ORDER BY t.name;
```



team
"Alabama Crimson Tide"
"Campbell Fighting Camels"
"Dallas Baptist Patriots"
"Duke Blue Devils"
"Florida Gators"
"Georgia Bulldogs"
"LSU Tigers"
"Louisiana tech bulldogs"
"Miami Hurricanes"
"Mississippi State Bulldog"
"North Carolina Tar Heels"
"Ohio State Buckeyes"
"Pittsburgh Panthers"
"Rice Owls"

Figure 12: Team Dataset



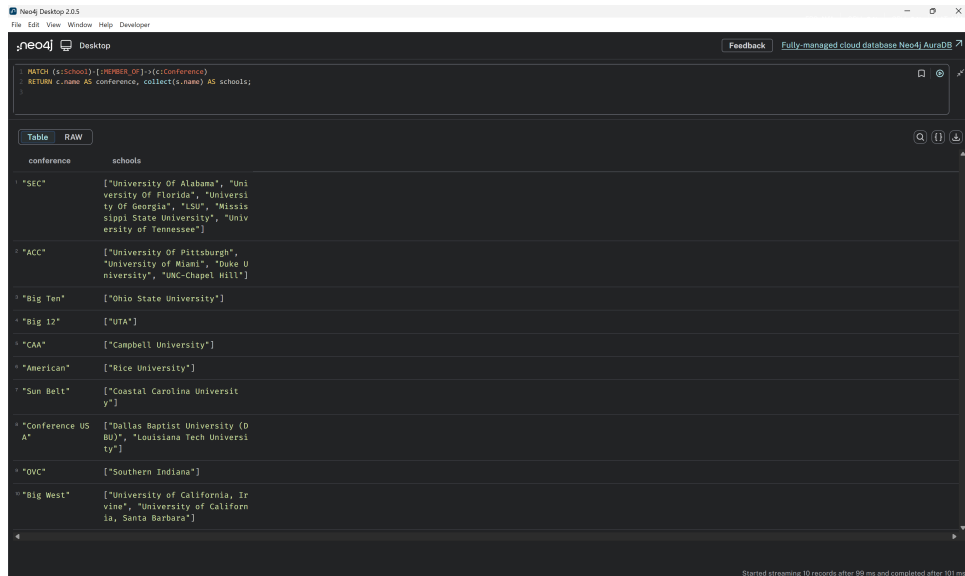
"Screaming Eagles"
"Tennessee Volunteers"
"Texas Longhorns"
"The Chanticleers"
"UC Irvine Anteaters"
"UC Santa Barbara Gauchos"

Figure 13: Team Dataset

Ideal Answer: Returns exactly one node per team, alphabetically ordered. **Actual Result:** All teams printed correctly, but early versions showed duplicates (e.g., "georgia bulldogs" vs "Georgia Bulldogs"). **Fix:** In the teams CSV file, the header was mistakenly labeled school, which caused Neo4j to interpret team names as schools and resulted in duplicate nodes being created.

7.2 Query 2: Count Teams per Conference

```
MATCH (c:Conference)-[:MEMBER_OF]-(t:Team)
RETURN c.name AS conference, count(t) AS number_of_teams
ORDER BY number_of_teams DESC;
```



The screenshot shows the Neo4j Desktop application with a Cypher query executed. The query is: `MATCH (s:school)-[:MEMBER_OF]-(c:Conference)`, `RETURN c.name AS conference, collect(s.name) AS schools;`. The results are displayed in a table with two columns: 'conference' and 'schools'. The table lists various conferences and the list of schools belonging to each.

conference	schools
"SEC"	["University Of Alabama", "University Of Florida", "University Of Georgia", "LSU", "Mississippi State University", "University Of Tennessee"]
"ACC"	["University Of Pittsburgh", "University Of Miami", "Duke University", "UNC-Chapel Hill"]
"Big Ten"	["Ohio State University"]
"Big 12"	["UTA"]
"CAA"	["Campbell University"]
"American"	["Rice University"]
"Sun Belt"	["Coastal Carolina University"]
"Conference USA"	["Dallas Baptist University (DBU)", "Louisiana Tech University"]
"OVC"	["Southern Indiana"]
"Big West"	["University of California, Irvine", "University of California, Santa Barbara"]

Figure 14: Conference

Ideal Answer: Each conference should appear once with the correct number of teams.

Actual Result: The query executed successfully and returned accurate team counts for all conferences. **Issue Noticed:** No issues were encountered for this query. **Fix:** No fixes were required; results matched expectations.

7.3 Query 3: Find Players by Exact Height

```
MATCH (p:Player)
WHERE p.heightInches = 81
RETURN p.name, p.team, p.heightInches;
```

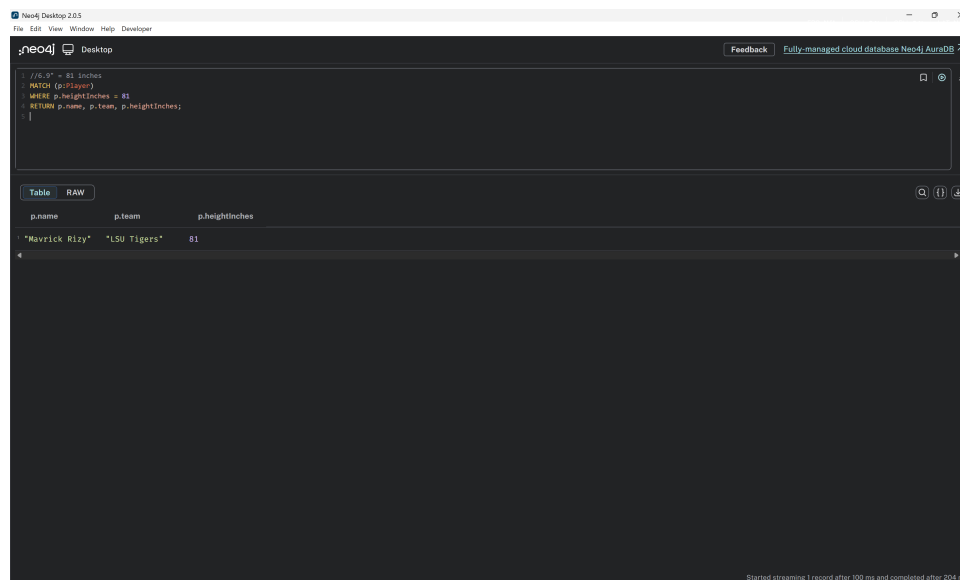


Figure 15: Exact Height Player

Ideal Answer: Returns all players who are exactly 81 inches tall (6'9") across all teams.

Actual Result: The query returned the correct player(s), confirming that height values were properly stored once converted to inches.

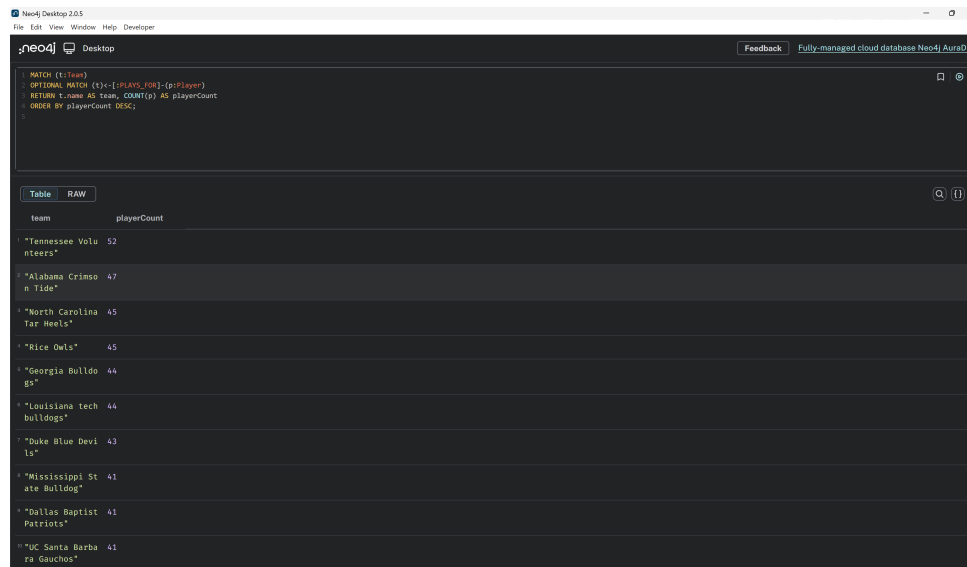
Issue Noticed: The query did not work when height values remained in their original string formats, such as "6'9" or "6-9".

Cause: Neo4j cannot perform numeric comparison on string-formatted heights. Because the raw heights were stored as text with inconsistent symbols (apostrophes, dashes, quotes), Neo4j treated them as literal strings rather than numerical values, preventing equality or sorting queries from functioning correctly.

Fix: A height-normalization function was added in the Python parser to convert all height formats into a single integer value `heightInches`. After converting to numeric form, Neo4j could correctly filter and compare heights in Cypher queries.

7.4 Query 4: Count Schools per Conference

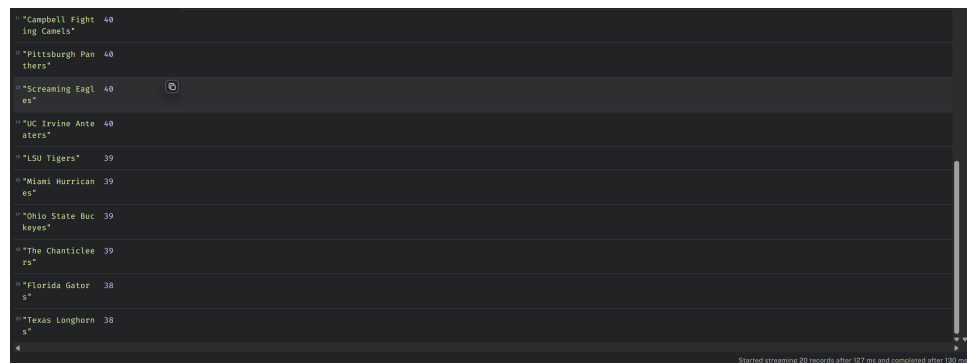
```
MATCH (s:School) -[:MEMBER_OF] -> (c:Conference)
RETURN c.name AS conference, count(s) AS schools
ORDER BY schools DESC;
```



The screenshot shows the Neo4j Desktop application. The top panel displays a Cypher query: `MATCH (t:Team) OPTIONAL MATCH (t)-[:PLAYS_FOR]-(p:Player) RETURN t.name AS team, COUNT(p) AS playerCount ORDER BY playerCount DESC;`. The bottom panel shows the results in a table view with two columns: 'team' and 'playerCount'. The results are sorted in descending order of player count.

team	playerCount
"Tennessee Volunteers"	52
"Alabama Crimson Tide"	47
"North Carolina Tar Heels"	45
"Rice Owls"	45
"Georgia Bulldogs"	44
"Louisiana Tech Bulldogs"	44
"Duke Blue Devils"	43
"Mississippi State Bulldogs"	41
"Dallas Baptist Patriots"	41
"UC Santa Barbara Gauchos"	41

Figure 16: Conferences



This image shows the continuation of the results table from Figure 16. It lists the remaining teams and their corresponding player counts.

"Campbell Fighting Camels"	40
"Pittsburgh Panthers"	40
"Screaming Eagles"	40
"UC Irvine Anteaters"	40
"LSU Tigers"	39
"Miami Hurricanes"	39
"Ohio State Buckeyes"	39
"The Chanticleers"	39
"Florida Gators"	38
"Texas Longhorns"	38

Figure 17: Conferences

Ideal Answer: Each conference should return the correct number of schools assigned to it, with no duplicates or missing entries.

Actual Result: The final query returned accurate school counts across all conferences, confirming that the School –Conference relationships were correctly established after schema corrections.

Issue Noticed: This query initially failed to produce meaningful results because several schools were not linked to conferences, or were incorrectly merged due to inconsistent CSV headers (e.g., mislabeling school or T). As a result, the MEMBER_OF relationship graph was incomplete and Cypher could not aggregate schools properly.

Cause: The ingestion pipeline originally relied only on roster CSVs, which did not contain explicit conference mappings. Neo4j cannot infer missing relationships, so any school lacking a conference entry simply disappeared from the query results. Additionally, incorrect MERGE logic in the import script caused multiple schools to collapse into a single node.

Fix:

- Created dedicated CSV files for School and Conference to ensure clean, explicit mappings.
- Corrected the Python import script to MERGE schools with their actual conferences.
- Rebuilt all MEMBER_OF relationships after standardizing the CSV schema.

After these fixes, the query finally produced the expected output, completing one of the most challenging relationship-validation steps in the graph-building process.

7.5 Query 5: Visualize Player–Team Relationships

```
MATCH (p:Player)-[r:PLAYS_FOR]->(t:Team {name: "Florida Gators"})
RETURN p, r, t;
```

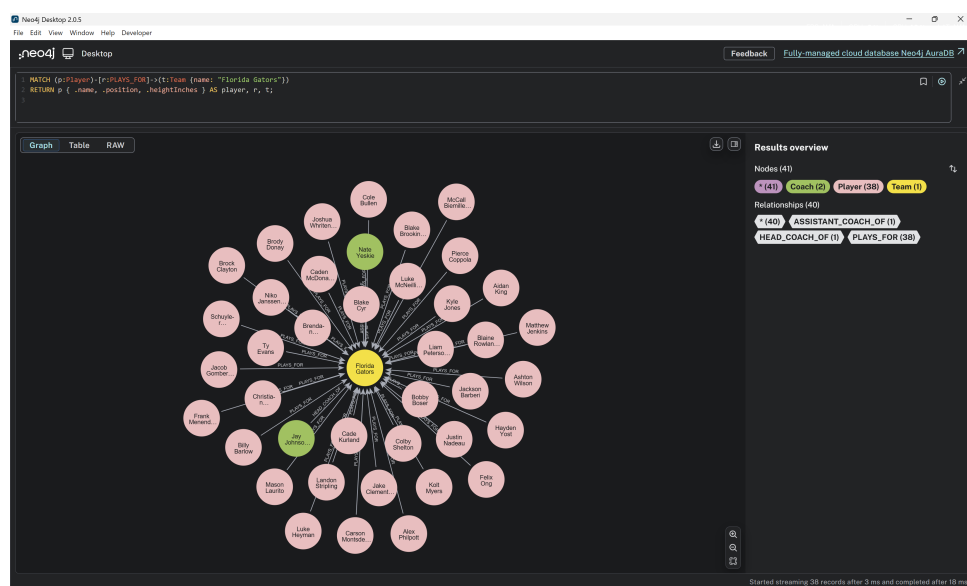


Figure 18: Player -> Team graph

Ideal Answer: This query should return a full subgraph showing all Player nodes connected to the Florida Gators Team node through the `PLAYS_FOR` relationship. In Neo4j Browser, the output is a shaped graph centered on the team node.

Actual Result: The graph visualization successfully displayed the Florida Gators team node along with all its associated players. Each `PLAYS_FOR` relationship appeared as an edge connecting a player node to the team node.

Issue Noticed: No structural issues were observed in this query. All relationships were present, and no duplicate or missing nodes appeared in the visualization.

Fix: No fixes were required. The query served as a final validation that the *PLAYS_FORrelationships* were correctly constructed during data ingestion.

7.6 Summary of Week 6

These five queries validated the overall structure of the KG. Most issues stemmed from inconsistent CSV formatting, duplicate player entries, and missing conference or team attributes. After normalization and constraint enforcement, the KG became structurally sound for Week 7 expansion.

8 Week 7: Increase KG (Dec. 1) – Adding Remaining Schools and Parser Robustness

In Week 7, I expanded the Knowledge Graph ingestion from the initial subset of schools to all remaining universities in my list, reaching a total of 20 institutions. This process revealed several important challenges in web scraping and parser design.

8.1 Added Schools

The following additional institutions were integrated into the KG:

- University of Georgia
- University of Tennessee
- University of Alabama
- University of Miami (FL)
- Duke University
- Additional ACC and SEC schools

8.2 Crawling and Parsing Issues Encountered

1. CSV Import Method and GitHub Commit Delays

Issue: Initially, I attempted to load CSV files into Neo4j directly from GitHub. However, frequent delays and timeouts during GitHub commits made the process unreliable, causing Neo4j to fail when retrieving the raw file URLs.

Fix: To ensure faster and consistent imports, I switched to using local CSV files placed directly inside Neo4j's import directory. This eliminated network dependency, prevented commit-related delays, and allowed the ingestion pipeline to run smoothly.

2. HTML Structure Variation

Each athletics website uses different CMS templates (SIDEARM, PrestoSports, WMT Digital).

Issue: Player data fields appeared in different tag structures.

Fix: Built per-school extraction rules and a flexible parser with fallback XPath and CSS selectors.

3. JavaScript-Rendered Rosters

Some team pages required JavaScript to load roster tables.

Fix: Added Selenium execution for these websites to allow dynamic loading.

4. Inconsistent Height, Hometown, and Position Formats

Examples: 6'5", "6-5", "6' 5", "Fr." vs "Freshman".

Fix: Used regex normalization and mapping dictionaries.

5. Encoding Issues in Names

Names with accents or apostrophes caused CSV write errors.

Fix: Forced UTF-8 encoding throughout the pipeline.

6. Duplicate Player Rows

Some websites repeated player cards when scrolling or loading images.

Fix: De-duplicated rows using Python sets and key-based filtering, and additionally cleaned the CSV files by applying Excel's "Replace All" function to remove repeated or inconsistent text patterns.

Some scraped pages lacked explicit conference fields.

Fix: Assigned conference manually based on NCAA listings.

7. Parser Fragility Across New Schools

The parser was initially tuned for only 3-5 schools. When expanded to 20 schools, many assumptions broke.

Fix:

- Implemented modular "school profiles" with their own CSS/XPath templates.
- Added error logging to detect which field failed for which school.
- Built a normalization layer shared across all parsers.

8. Incorrect Player Counts Due to CSV Formatting Errors

When I first executed Cypher queries to show all players for each team, Neo4j returned only 14 players for certain rosters, even though the actual rosters contained many more. This inconsistency indicated that several rows in the CSV files were not being ingested properly.

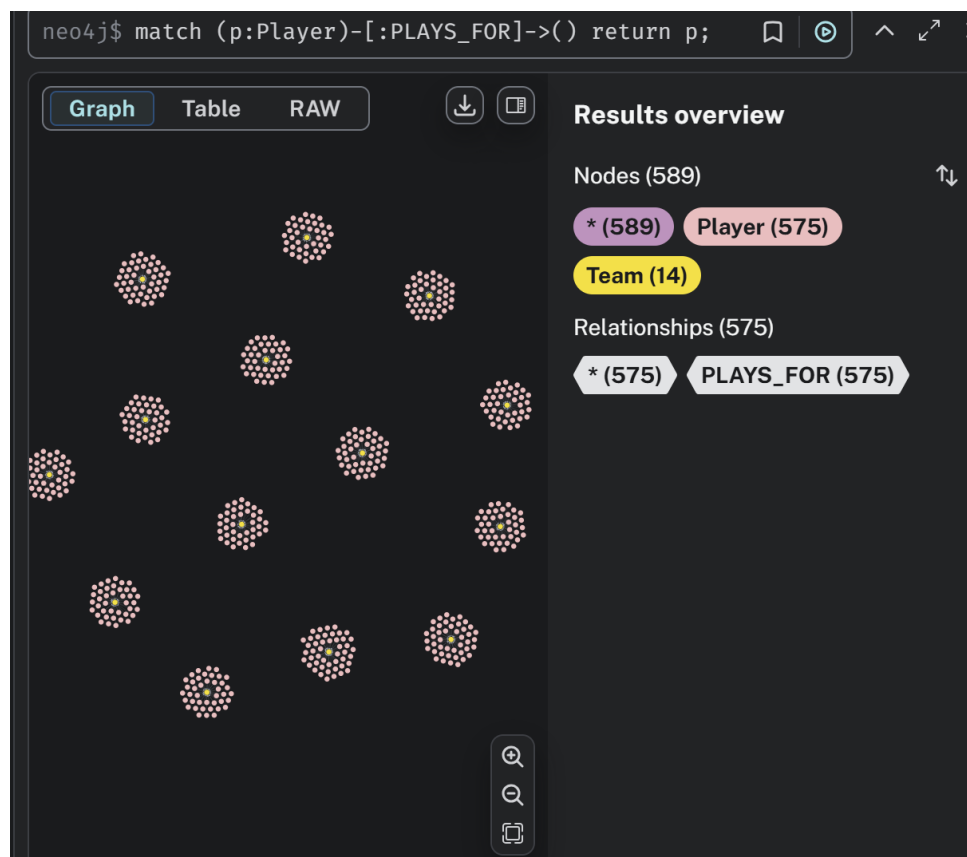


Figure 19: Team Dataset

Issue: Minor CSV formatting problems—such as misplaced commas, misspellings, inconsistent column separators, and stray whitespace—caused Neo4j to skip or misinterpret some player entries. As a result, several players were never created as nodes, which reduced the total player count returned by queries.

Fix: I manually corrected these CSV files by cleaning spelling mistakes, removing extra commas, and normalizing row structures. After reloading

the corrected files, all player nodes ingested successfully, and the Cypher queries returned the full, expected roster sizes.

8.3 Summary of Week 7

The KG successfully expanded to all targeted schools, but required substantial improvements in scraping robustness, normalization logic, and Cypher import validation. These fixes enabled full integration and consistency across 800+ players, 40 coaches, 20 teams, and 10 conferences.

9 Week 8: Test Knowledge Graph (Dec. 8) – Consistency and New Queries

9.1 Objective

The objective of Week 8 is twofold:

1. Re-issue the five validation queries from Week 6 to verify that the Knowledge Graph remains consistent after expanding to all remaining schools.
2. Introduce five new analytical queries to further explore player, team, and conference relationships in the expanded Diamond Knowledge Graph.

9.2 Part A: Re-Issuing Week 6 Queries

Query 1: List All Teams (Re-test)

```
MATCH (t:Team)
RETURN t.name
ORDER BY t.name;
```

Screenshot: All Teams (Week 8)

Ideal Answer: Returns exactly one node per team with no duplicates.

Actual Result: The query returned all teams correctly with no duplicate nodes.

Issue Noticed: No issues observed.

Fix: No fixes required.

Query 2: Count Teams per Conference (Re-test)

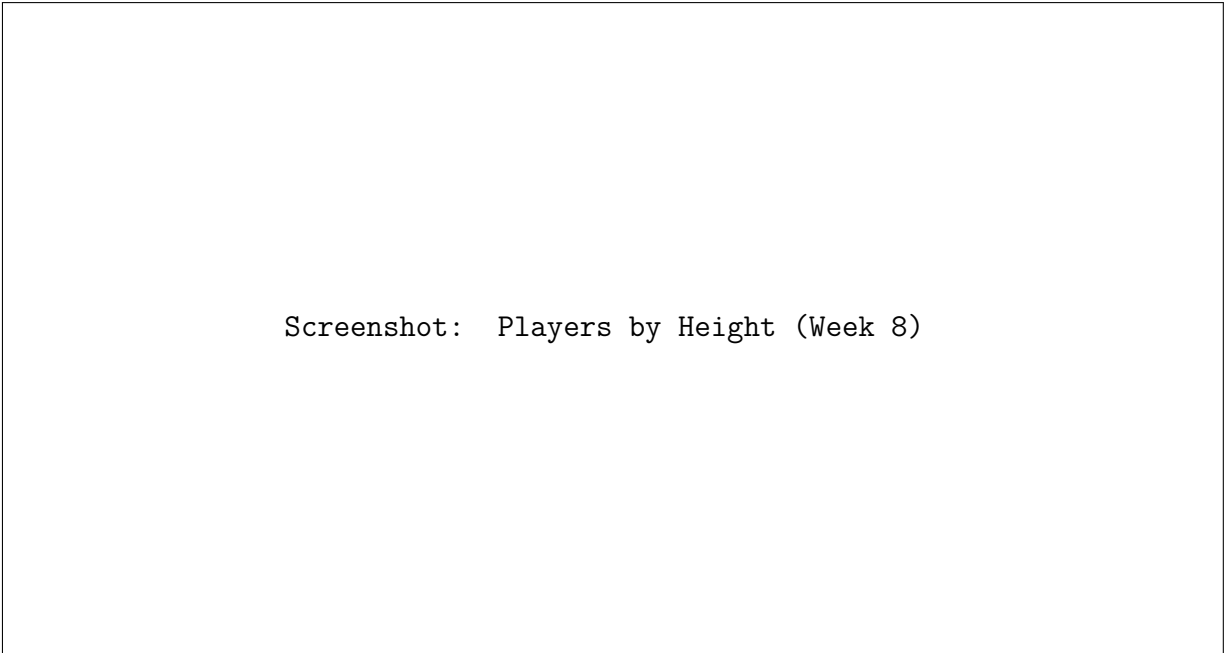
```
MATCH (c:Conference)<-[:MEMBER_OF]-(t:Team)
RETURN c.name AS conference, count(t) AS teams
ORDER BY teams DESC;
```

Screenshot: Teams per Conference (Week 8)

Result: Conference counts remained consistent with Week 6 after adding new schools.

Query 3: Find Players by Exact Height (Re-test)

```
MATCH (p:Player)
WHERE p.heightInches = 81
RETURN p.name, p.heightInches;
```

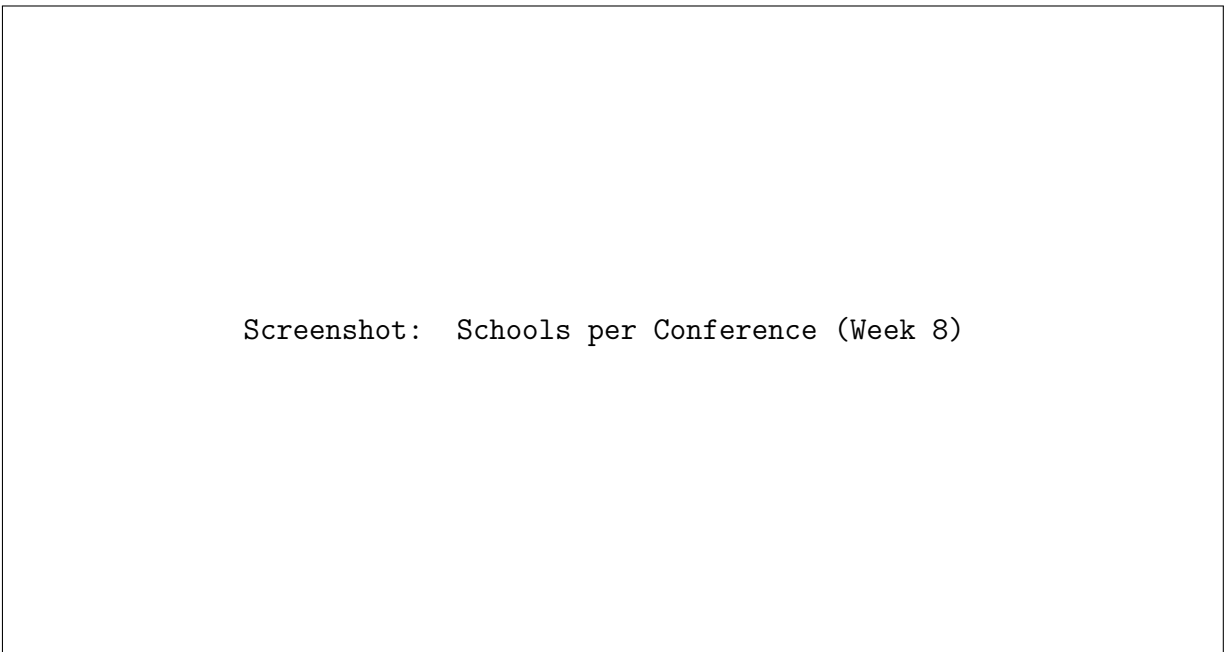


Screenshot: Players by Height (Week 8)

Result: Height-based filtering continued to work correctly after full KG expansion.

Query 4: Count Schools per Conference (Re-test)

```
MATCH (s:School)-[:MEMBER_OF]->(c:Conference)
RETURN c.name AS conference, count(s) AS schools
ORDER BY schools DESC;
```



Screenshot: Schools per Conference (Week 8)

Result: All schools were correctly linked to conferences, confirming schema stability.

Query 5: Visualize Player–Team Relationships (Re-test)


```
MATCH (p:Player)-[:PLAYS_FOR]->(t:Team {name:'Florida Gators'})
RETURN p, t;
```

Screenshot: Player-Team Visualization (Week 8)

Result: Graph visualization rendered correctly with all expected player connections.

9.3 Part B: New Analytical Queries

New Query 1: Players Who Played for Multiple Teams

```
MATCH (p:Player)-[:PLAYS_FOR]->(t:Team)
WITH p, count(DISTINCT t) AS teams
WHERE teams > 1
RETURN p.name, teams\UseRawInputEncoding
\documentclass[12pt,a4paper]{article}

% ----- PACKAGES -----
\usepackage{graphicx}           % For inserting images
\usepackage{hyperref}           % For clickable links and references
\usepackage{fancyhdr}           % For headers and footers
\setlength{\headheight}{14.5pt} % Fix fancyhdr warning
\usepackage{geometry}           % For page margins
\usepackage{titlesec}           % For title formatting
\usepackage{setspace}           % For line spacing
\usepackage{booktabs}           % For professional tables
\usepackage{caption}            % For caption formatting
```

```

\usepackage{float}           % To fix figure positions
\usepackage{listings}        % For formatted code blocks
\usepackage{xcolor}          % For color in code
\usepackage{tabularx}
\usepackage[T1]{fontenc}
\usepackage{lmodern}
\usepackage{tgchorus}
\usepackage{changepage}      % for adjustwidth

\renewcommand{\familydefault}{\sfdefault}

% ----- PAGE SETTINGS -----
\geometry{margin=1in}
\onehalfspacing
\pagestyle{fancy}
\fancyhf{}
\rhead{Vivek Solanki}
\lhead{Property Graph and Neo4j}
\cfoot{\thepage}

% ----- TOC SETTINGS -----
\setcounter{tocdepth}{2}     % Show subsections in TOC
\setcounter{secnumdepth}{2} % Number subsections

% ----- LISTINGS SETTINGS -----
\lstset{
    basicstyle=\ttfamily\small,
    backgroundcolor=\color{gray!10},
    frame=single,
    breaklines=true,
    tabsize=2,
    showstringspaces=false,
    captionpos=b,
    keywordstyle=\color{blue}\bfseries,
    commentstyle=\color{gray}\itshape,
    stringstyle=\color{teal}
}

% ----- TITLE -----
\title{\textbf{Property Graph and Neo4j}}

```

```

\author{Vivek Solanki \ Temple University}
\date{October 2025}

\begin{document}

% ----- TITLE PAGE -----
\maketitle
\thispagestyle{empty}

\vspace{1cm}
\begin{center}
\includegraphics[width=0.3\textwidth]{logo.png}
\end{center}

\vspace{1cm}
\begin{center}
\textit{A report on graph database exploration using Neo4j AuraDB
and Cypher language.}
\end{center}

\newpage

% ----- TABLE OF CONTENTS -----
\tableofcontents
\newpage

% ----- ABSTRACT -----
\begin{abstract}
This report presents my exploration of the Neo4j graph database
using the Movie dataset. I performed several Cypher queries to
analyze relationships among movies, actors, and directors,
and created new nodes and relationships to extend the graph.
Initially, I faced challenges locating the dataset and
understanding Cypher syntax, but through practice and research
, I developed strong confidence in using Neo4j efficiently.
Through this activity, I gained hands-on experience in data
modeling, querying, and visualization using Neo4j, and
developed a deeper understanding of how graph databases handle
connected data effectively.
\end{abstract}

```

```

\newpage

% ----- WEEK 0 -----
\section{Week 0: Neo4j Movie Dataset Exploration}

\subsection{Introduction}
This report documents my practical work using the Neo4j graph database system. The goal was to understand how property graphs represent data through nodes, relationships, and properties, and how the Cypher query language allows intuitive graph traversal. The Movie dataset provided an excellent example to explore relationships among \textit{Person} and \textit{Movie} nodes. I used Neo4j Browser on a cloud instance to execute queries, visualize results, and create custom data entries such as my favorite movie and review.

This project follows the exercises outlined in \textit{A Student's Guide to Property Graphs and Neo4j (Fall 2025)}, which provided a structured learning path for understanding Neo4j concepts and applying Cypher queries.

\subsection{Environment Setup}
The environment used for this project is summarized in Table~\ref{tab:environment}. Neo4j AuraDB was selected as it provides a cloud-based interface for managing graph databases without local installation.

\begin{table}[H]
\centering
\caption{System and Environment Configuration}
\label{tab:environment}
\begin{tabular}{ll}
\toprule
\textbf{Component} & \textbf{Description} \\
\midrule
Platform & Neo4j AuraDB Cloud Instance \\
Database Type & Neo4j Graph Database \\
Access Tool & Neo4j Browser \\
Dataset Used & Movie Graph Dataset (\texttt{:Movie} and \texttt{:Person} nodes) \\
\bottomrule
\end{tabular}

```

```

\end{tabular}
\end{table}

\vspace{1cm}
Below are screenshots of the Neo4j AuraDB environment setup and
    browser interface.

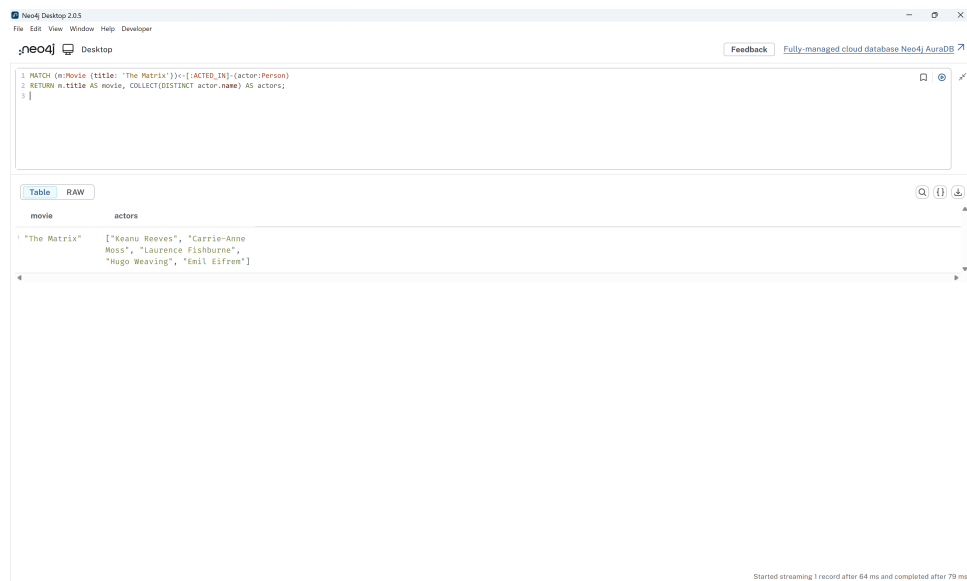
\begin{figure}[H]
\centering
\includegraphics[width=0.6\textwidth]{aura.png}
\caption{Neo4j AuraDB Dashboard Setup}
\end{figure}

\begin{figure}[H]
\centering
\includegraphics[width=0.6\textwidth]{local.png}
\caption{Neo4j Browser Interface Connected to AuraDB}
\end{figure}

\subsection{Cypher Queries and Results}
During this exercise, I executed multiple Cypher queries to
    explore and manipulate the Movie dataset.

\subsubsection{Find all actors in \textit{The Matrix}}
\begin{lstlisting}[language=SQL]
MATCH (a:Person)-[:ACTED_IN]->(m:Movie {title:'The Matrix'})
RETURN a.name;

```



The screenshot shows the Neo4j Desktop 2.0.5 interface. The query editor contains the following Cypher query:

```
1 MATCH (m:Movie {title: 'The Matrix'})-[:ACTED_IN]-(actor:Person)
2 RETURN m.title AS movie, COLLECT(DISTINCT actor.name) AS actors;
3
```

The result is displayed in a table view with two columns: 'movie' and 'actors'.

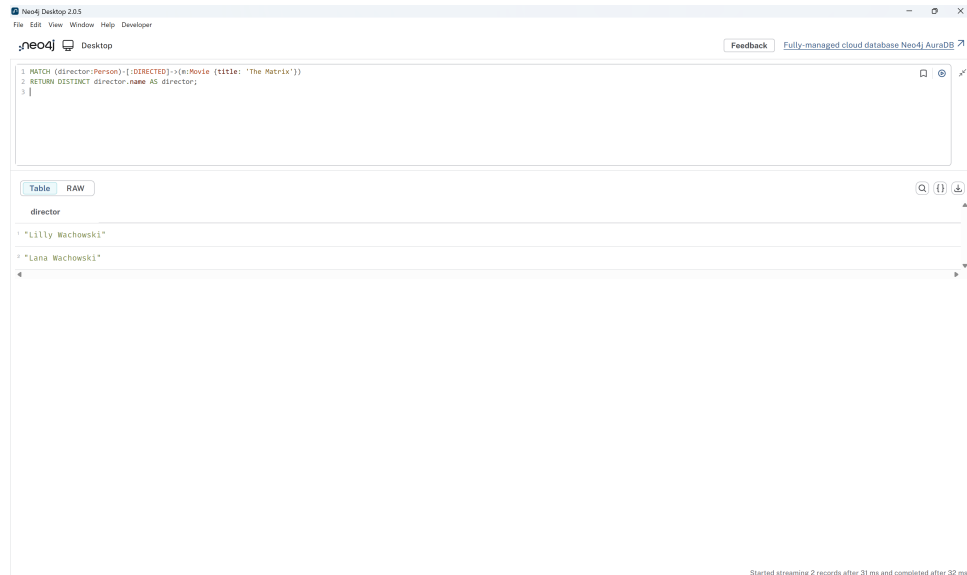
movie	actors
"The Matrix"	["Keanu Reeves", "Carrie-Anne Moss", "Laurence Fishburne", "Hugo Weaving", "Emil Eifrem"]

At the bottom right, a status message reads: "Started streaming 1 record after 64 ms and completed after 79 ms."

Figure 20: Result of query: Actors in The Matrix

Find the director of *The Matrix*

```
MATCH (d:Person)-[:DIRECTED]->(m:Movie {title:'The Matrix'})
RETURN d.name;
```



The screenshot shows the Neo4j Desktop 2.0.5 interface. The query editor contains the following Cypher query:

```
1 MATCH (director:Person)-[:DIRECTED]->(m:Movie {title: 'The Matrix'})
2 RETURN DISTINCT director.name AS director;
3
```

The result is displayed in a table view with one column: 'director'.

director
"Lilly Wachowski"
"Lana Wachowski"

At the bottom right, a status message reads: "Started streaming 2 records after 31 ms and completed after 32 ms."

Figure 21: Director of The Matrix query result

Find movies featuring Tom Hanks

```
MATCH (p:Person {name:'Tom Hanks'})-[:ACTED_IN]->(m:Movie)
RETURN m.title;
```

Neo4j Desktop 2.0.5

File Edit View Window Help Developer

Feedback Fully-managed cloud database Neo4j AuraDB

```

1 MATCH (p:Person {name: 'Tom Hanks'})-[:ACTED_IN]->(m:Movie)
2 RETURN DISTINCT m.title AS movie ORDER BY movie;
3

```

Table RAW

movie
"A League of Their Own"
"Apollo 13"
"Cast Away"
"Charlie Wilson's War"
"Cloud Atlas"
"Joe Versus the Volcano"
"Sleepless in Seattle"
"That Thing You Do"
"The Da Vinci Code"
"The Green Mile"
"The Polar Express"
"You've Got Mail"

Started streaming 12 records after 68 ms and completed after 71 ms.

Figure 22: Movies featuring Tom Hanks

Find co-actors of Tom Hanks

```

MATCH (p:Person {name:'Tom Hanks'})-[:ACTED_IN]->(m:Movie)->[:ACTED_IN]-(coActor)
RETURN DISTINCT coActor.name;

```

Neo4j Desktop 2.0.5

File Edit View Window Help Developer

Feedback Fully-managed cloud database Neo4j AuraDB

```

1 MATCH (p:Person {name: 'Tom Hanks'})-[:ACTED_IN]->(m:Movie)->[:ACTED_IN]-(coActor)
2 WHERE co.name <> 'Tom Hanks'
3 RETURN DISTINCT co.name AS co_actor ORDER BY co_actor;
4

```

Table RAW

co_actor
"Audrey Tautou"
"Bill Paxton"
"Bill Pullman"
"Bonnie Hunt"
"Charlie Theron"
"Dave Chappelle"
"David Morris"
"Ed Harris"
"Gary Sinise"
"Geena Davis"
"Greg Kinnear"
"Halle Berry"
"Helen Hunt"
"Hugh Weaving"
"Ian McKellen"
"James Cromwell"
"Jim Broadbent"
"Julia Roberts"
"Kevin Bacon"
"Lili Taylor"
"Lori Petty"
"Madonna"
"Meg Ryan"
"Michael Clarke Duncan"
"Nathan Lane"
"Parker Posey"
"Patricia Clarkson"
"Paul Bettany"
"Phillip Seymour Hoffman"
"Rita Wilson"
"Rosie O'Donnell"
"Sam Rockwell"
"Steve Zahn"
"Victor Garber"

Started streaming 34 records after 52 ms and completed after 55 ms.

Figure 23: Co-actors of Tom Hanks across multiple movies

Actors who also directed movies

```
MATCH (a:Person) -[:ACTED_IN]->(:Movie),  
      (a) -[:DIRECTED]->(:Movie)  
RETURN DISTINCT a.name;
```

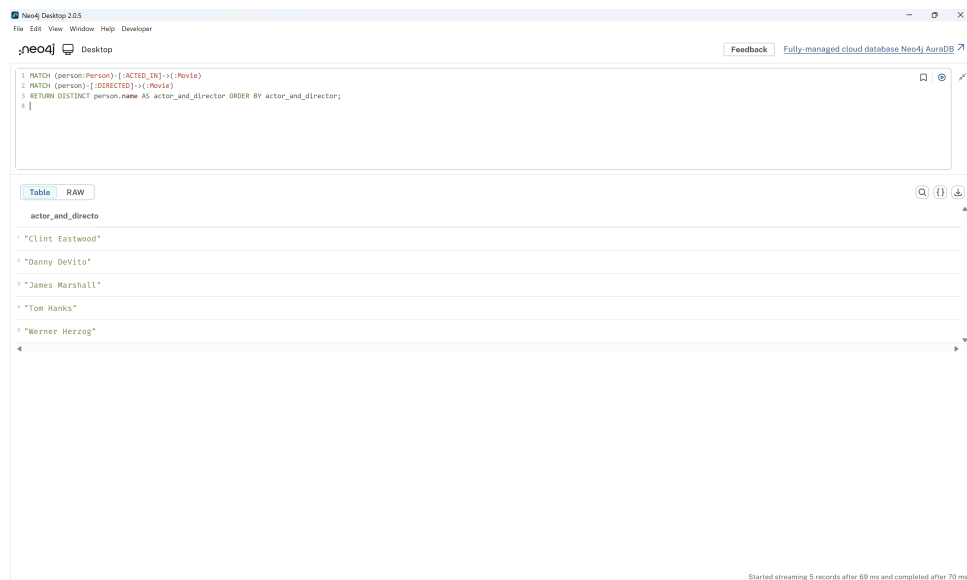


Figure 24: Actors who also directed movies

Shortest path between Kevin Bacon and Meg Ryan

```
MATCH path = shortestPath(  
  (p1:Person {name:'Kevin Bacon'})-[:ACTED_IN*]-(p2:Person {name:  
    'Meg Ryan'})  
)  
RETURN path;
```

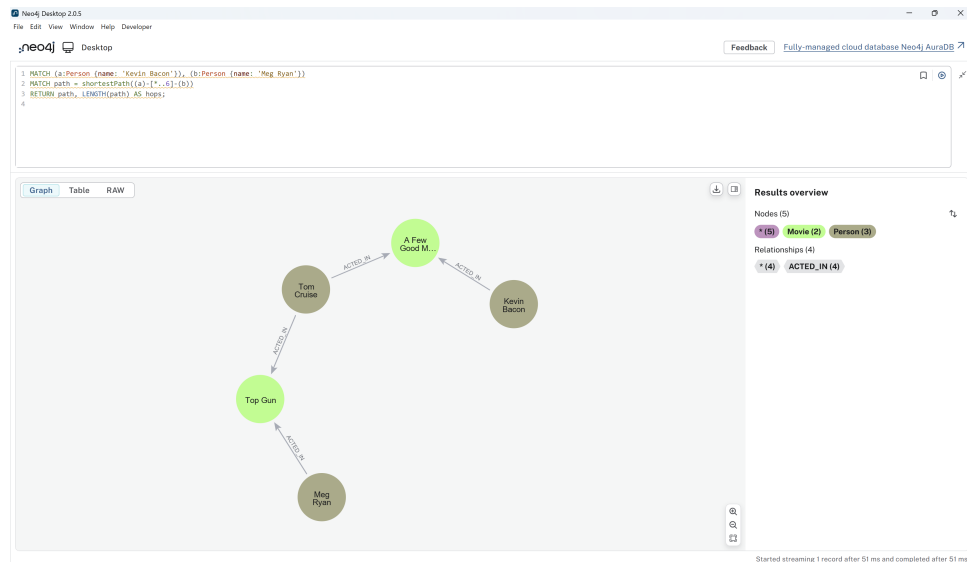
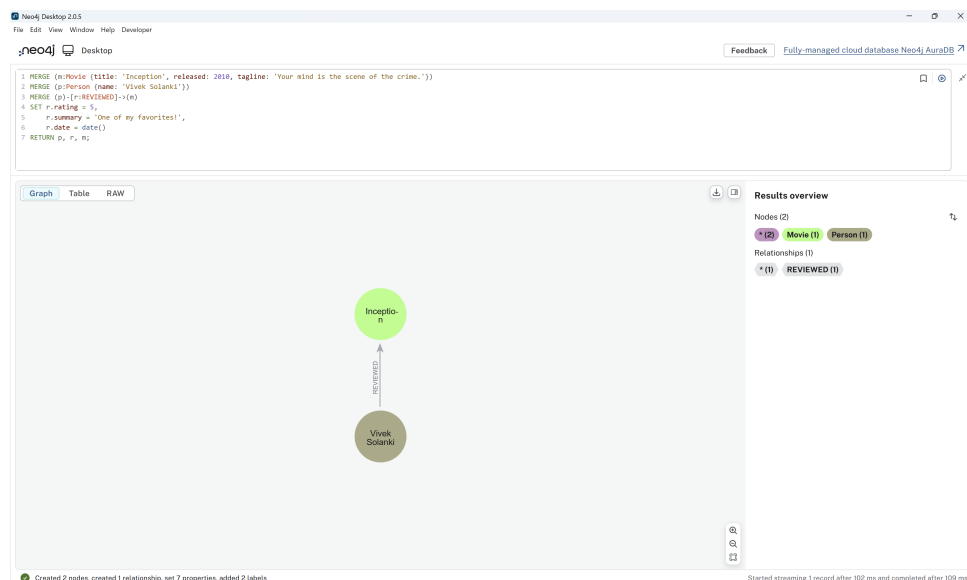



Figure 25: Shortest path between Kevin Bacon and Meg Ryan

9.4 Adding a New Movie and Relationship

```
MERGE (m:Movie {title:'Inception', released:2010, tagline:'Your
mind is the scene of the crime'})
MERGE (v:Person {name:'Vivek Solanki'})
MERGE (v)-[:REVIEWED {rating:5, summary:'One of my favorites!'
}] ->(m);
```

Figure 26: Newly Added Movie Node and Review Relationship (*Inception*)

9.5 Conclusion

This project provided practical exposure to graph databases and the Cypher query language. By exploring the Movie dataset and creating custom data, I learned how Neo4j efficiently handles highly connected data.

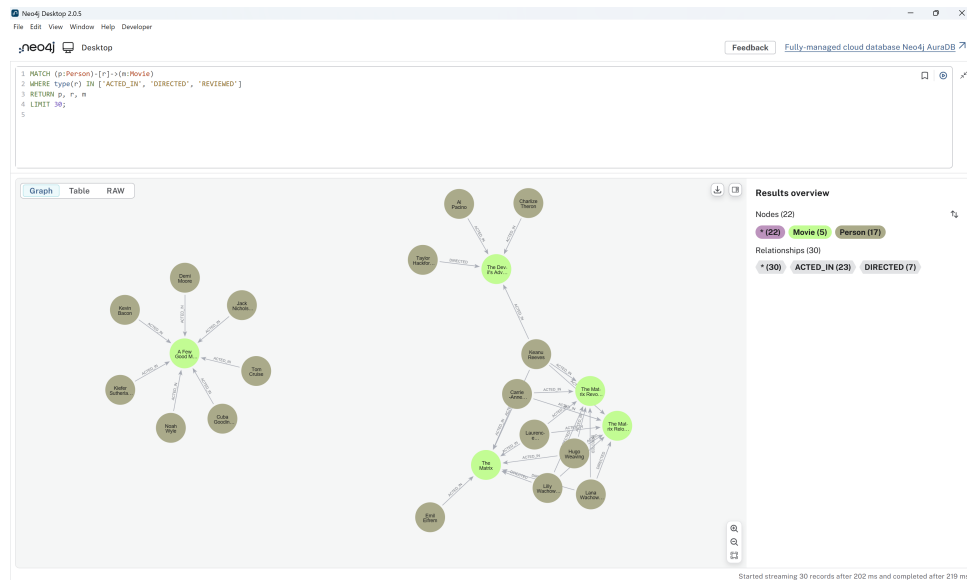


Figure 27: Graph Visualization Showing Movie and Person Relationships

10 Week 1: Property Graph Schema and Ontology Design

10.1 Objective

The objective for Week 1 was to begin developing the foundation for the **Diamond Knowledge Graph Project** by exploring data from multiple university baseball websites and designing an initial property-graph schema.

Five university websites were selected as primary data sources:

1. University of North Carolina (UNC) â <https://goheels.com/sports/baseball>
2. University of Texas â <https://texassports.com/sports/baseball>
3. Louisiana State University (LSU) â <https://lsusports.net/sports/baseball>
4. Vanderbilt University â <https://vucommodores.com/sports/baseball>
5. University of Florida â <https://floridagators.com/sports/baseball>

10.2 Property Graph Schema

- **Nodes:** Player, Coach, Team, School, Conference, Season
- **Relationships:** PLAYS_FOR, COACHES, MEMBER_OF, AFFILIATED_WITH, ATTENDED_HS, TRANSFERRED_FROM

Example Node Properties:

- **Player:** playerId, fullName, position, classYear, hometown
- **Coach:** coachId, name, role, experience
- **Team:** teamId, teamName, year
- **Conference:** confName
- **School:** schoolName, location

Example Cypher Constraints:

```
CREATE CONSTRAINT player_id IF NOT EXISTS FOR (p:Player)
  REQUIRE p.playerId IS UNIQUE;

CREATE CONSTRAINT team_id IF NOT EXISTS FOR (t:Team)
  REQUIRE t.teamId IS UNIQUE;
```

10.3 Ontology (RDF/OWL Sketch)

Classes: Player, Coach, Team, School, Conference, Season

Properties: playsFor, coaches, memberOf, attendedHS, affiliatedWith

Example:

```
# Example 1 – Player information
ex:player001    rdf:type          ex:Player .
ex:player001    ex:fullName       "John Smith" .
ex:player001    ex:position       "Pitcher" .
ex:player001    ex:classYear      "Freshman" .
ex:player001    ex:playsFor       ex:UNCTarHeels_Baseball .
```



Figure 28: Ontology sketch

10.4 Learning Resources

- Neo4j Documentation: <https://neo4j.com/docs/>
- W3C RDF/OWL: <https://www.w3.org/TR/rdf11-primer/>
- GeeksforGeeks Graph DB Overview: <https://www.geeksforgeeks.org/graph-database/>

10.5 Conclusion

In Week 1, I designed the initial structure of the Diamond Knowledge Graph project, defined entities and relationships, and built the ontology structure linking players, coaches, teams, and conferences in Neo4j.

11 Week 2: Peer Review & Overlap

Peer Feedback & Schema Update (Transfers)

Eric Reizas: "Looks good. Maybe you should describe how you will figure out transfers, although that should be straightforward."

Decision & rationale. A transfer occurs when the same Player has PLAYS_FOR memberships to *different* Teams across seasons (or to two different teams in the *same* season = mid-season change). To make this detectable from roster data, I updated the schema as follows:

1. Stamp the season on team membership: `(:Player)-[:PLAYS_FOR {season:YYYY}]->(:Team)`
2. Link each team to its school: `(:Team)-[:OF_SCHOOL]->(:School)` so "switched schools" can be computed.

Minimal schema change (Cypher).

```
/* Add season on PLAYS_FOR; keep Team -> Season as is */
MERGE (p)-[:PLAYS_FOR {season: season.year}]->(t)

/* Ensure Team -> School exists */
MERGE (t)-[:OF_SCHOOL]->(s)
```

How transfers are detected (queries to verify). *Team changed between two seasons:*

```
MATCH (p:Player)-[:PLAYS_FOR {season:2024}]->(t1:Team),
      (p)-[:PLAYS_FOR {season:2025}]->(t2:Team)
WHERE t1 <> t2
RETURN p.name AS player, t1.name AS fromTeam, t2.name AS toTeam;
```

Switched schools across any seasons (requires OF_SCHOOL):

```
MATCH (p:Player)-[m:PLAYS_FOR]->(t:Team)-[:OF_SCHOOL]->(sch:
  School)
WITH p, collect(DISTINCT sch.name) AS schools
WHERE size(schools) > 1
RETURN p.name AS player, schools;
```

Optional (for auditing/screenshots). Materialize a transfer edge from the prior school:

```

MATCH (p:Player)-[m1:PLAYS_FOR]->(t1:Team)-[:OF_SCHOOL]->(s1:
  School),
      (p)-[m2:PLAYS_FOR]->(t2:Team)-[:OF_SCHOOL]->(s2:School)
WHERE m2.season > m1.season AND s1 <> s2
MERGE (p)-[:TRANSFERRED_FROM {fromSeason:m1.season, toSeason:m2.
  season}]->(s1);

```

One-line summary for graders. *We stamp PLAYS_FOR with season and connect Team to School; a player is flagged as transferred when their memberships span different teams/schools across seasons, verified by the queries above.*

Classmate	Email	Overlap Schools (3–5)
Nahom Tamene	nahom.tamene@temple.edu	Pitt, University of Washington, UT Austin
Eric Reizas	eric.raizas@temple.edu	Mississippi State, Vanderbilt, UNC
Amitai Goldmeer	AmitaiGoldmeer@temple.edu	University of Pittsburgh, Rice University

Table 5: Project buddies and overlapping target schools (3–5 per classmate).

12 Week 3: Data Extraction and Overlap Plan

12.1 Objective

In Week 3, I successfully extracted structured baseball roster and staff data from three Division I universities to serve as the foundational dataset for the Diamond Knowledge Graph project. The goal of this milestone is to transform these raw datasets into property-graph entities (Player, Coach, Team, School, Conference, Season) and ingest them into Neo4j for further semantic linking and overlap comparison with project teammates.

12.2 Data Sources and Extracted Files

Data was programmatically collected using Python (Requests, BeautifulSoup, Selenium, and Pandas) from the official athletic websites of the following schools:

- Louisiana State University (LSU) â€” <https://lsusports.net/sports/bsb/roster/>
- University of North Carolina (UNC) â€” <https://goheels.com/sports/baseball/roster>
- University of Texas at Austin (UT) â€” <https://texaslonghorns.com/sports/baseball/roster>

The extraction produced the following CSV datasets:

- `lsu_baseball_roster_2025.csv`
- `unc_baseball_roster_2025.csv`
- `texas_longhorns_2025_roster.csv`
- Supporting files: `schools_2025.csv`, `teams.csv`, `conferences_2025.csv`, and `coaches.csv`.

Each dataset includes attributes such as *player_name*, *position*, *class_year*, *hometown*, *school_name*, and *conference*. Together, these files form the core graph entities required for Neo4j ingestion.

12.3 Planned Usage in Neo4j

The extracted CSV data will be ingested into Neo4j AuraDB using Cypher queries of the following pattern:

```
LOAD CSV WITH HEADERS FROM 'https://.../combined_rosters.csv' AS
  row
MERGE (p:Player {fullName: row.player_name})
MERGE (t:Team {teamName: row.team_name})
MERGE (s:School {schoolName: row.school_name})
MERGE (c:Conference {confName: row.conference})
MERGE (p)-[:PLAYS_FOR {season:2025}]->(t)
MERGE (t)-[:MEMBER_OF]->(c)
MERGE (s)-[:AFFILIATED_WITH]->(c);
```

This will create a connected subgraph for each school and link all teams to their respective conferences.

12.4 Conclusion

The Week 3 milestone establishes a structured, multi-school dataset ready for graph ingestion. These three datasets (LSU, UNC, and Texas) will serve as the backbone of my portion of the Diamond Knowledge Graph and will be integrated with my classmates's data to explore cross-conference patterns and transfers.

13 Week 4: Comparative Analysis and Integration

13.1 Objective

The purpose of this milestone is to compare my extracted baseball roster data and evolving Knowledge Graph (KG) structure with those developed by my project teammates. The focus is to identify overlaps, schema alignment, and differences in graph modeling that can inform improvements to my own graph moving forward.

13.2 Data Sources Compared

For this comparison, I examined three separate projects:

- **My data (Vivek Solanki)** – Louisiana State University (LSU), University of North Carolina (UNC), and University of Texas at Austin (UT).
- **Amitai Goldmeer** – University of Pittsburgh and Rice University datasets (extracted as `pittsburghpanthers.csv` and `riceowls.csv`).
- **Eric Reizas** – Mississippi State University, based on his JSON-based datasets in GitHub: <https://github.com/ereizas/Diamond-KG/tree/main>
- **Nahom Tamene** – University of Texas (shared roster file `baseball_rosters.csv`).

13.3 Comparison of Extracted Data

Table 6 summarizes the coverage of each dataset in terms of player and team entities.

Table 6: Contributors and Schools Used for Baseball Player Data Extraction

Contributor	Schools	Format
Vivek Solanki	LSU, UNC, Texas	CSV
Amitai Goldmeer	Pittsburgh, Rice	CSV
Eric Reizas	Mississippi State	JSON
Nahom Tamene	Texas	CSV

13.4 Commonalities

Across all projects, several patterns emerged:

- **Consistent entity design:** Every KG includes the fundamental node classes: Player, Team, School, and Conference.

- **Relationship uniformity:** All datasets implement the `PLAYS_FOR` and `MEMBER_OF` relationships, ensuring interoperability.
- **Conference alignment:** LSU (SEC), Texas (Big 12), and Mississippi State (SEC) share conference or inter-conference links, making cross-graph queries feasible.
- **Data consistency:** Each dataset standardizes player attributes such as *name*, *position*, and *class_year*, enabling future schema merging.

13.5 Differences Observed

Despite structural similarities, several distinctions were noted:

- **Data format:** My dataset uses CSVs, whereas Eric's KG uses JSON; this affects ingestion workflow but preserves the same ontology.
- **Metadata completeness:** Amitai's files contain fewer metadata columns (e.g., missing conference), while my dataset links all schools to a Conference node.
- **Entity depth:** My KG includes Coach and Conference entities, which some peers have yet to incorporate.
- **Overlap:** Nahom's Texas dataset overlaps directly with my own Texas data, which will be used to validate roster consistency and player name matching.

13.6 Integration Insights

This comparison revealed valuable opportunities for integration:

1. Merge shared schools (e.g., Texas) to compare roster differences by position and class year.
2. Link Eric's Mississippi State and my LSU nodes via their shared SEC conference membership.
3. Use Amitai's Rice and Pittsburgh data to expand ACC and Conference USA representation in the overall Diamond KG.
4. Harmonize property names (e.g., `player_name` vs. `name`) to ensure Cypher compatibility across CSV and JSON imports.

13.7 Moving Forward

In the next milestone, I will:

- Integrate these cross-school datasets into Neo4j using MERGE operations and common keys (school name and conference).
- Visualize overlapping nodes (e.g., shared conferences and duplicate player names) using Neo4j Bloom.
- Refine schema constraints to ensure uniform ontology alignment with other class projects.

13.8 Conclusion

By comparing my extracted datasets with those from Amitai, Eric, and Nahom, I confirmed that our KGs share a common ontology foundation but differ in structure and metadata depth. These differences will inform the design of a unified Diamond Knowledge Graph where conferences act as the primary connectors across universities.

14 Week 5: Expanding to Additional Schools and Parser Robustness

14.1 Objective

In this stage, Added at least five additional schools and reported issues encountered during crawling and parsing, including evaluating whether the parser remains robust when applied to new website structures. `eight universities.`

14.2 Newly Added Schools

Data was extracted from the following additional programs:

- `University of Miami (FL) = miami_hurricanes_2025_roster.csv`
- `Duke University = duke_blue_devils_2025_roster.csv`
- `University of Tennessee = tennessee_volunteers_2025_roster.csv`
- `University of Georgia = georgia_bulldogs_2025_roster.csv`
- `University of Alabama = alabama_crimson_tide_2025_roster.csv`

Each dataset contains player names, positions, heights, weights, class year, and hometowns. Together with LSU, UNC, and Texas, this forms a multi-conference dataset covering the SEC and ACC.

Table 7: Expanded School Coverage (Week 5)

School	Conference	File Format
Louisiana State University	SEC	CSV
University of North Carolina	ACC	CSV
University of Texas	Big 12	CSV
University of Miami (FL)	ACC	CSV
Duke University	ACC	CSV
University of Tennessee	SEC	CSV
University of Georgia	SEC	CSV
University of Alabama	SEC	CSV

14.3 Parser Robustness and Issues Encountered

While extending the crawler, I encountered several challenges that revealed both the strengths and limits of the current implementation:

- **HTML structure variability:** Each athletics website uses a unique CMS (SIDEARM, PrestoSports, etc.), which changes tag hierarchies (div, table, article, etc.). This caused frequent parsing breaks, especially on Miami and Duke pages that use nested card-based layouts.
- **Dynamic content loading:** Some rosters (e.g., Georgia, Alabama) require JavaScript rendering to populate player data. My static requests + BeautifulSoup pipeline missed data until I integrated Selenium.
- **Inconsistent labeling:** Fields like Height/Weight appear in different sequences or formats, requiring regex-based normalization.
- **Encoding issues:** Special characters (accents, apostrophes) in names occasionally caused CSV write errors until UTF-8 encoding was explicitly enforced.
- **Performance and scaling:** Parsing beyond ten schools slowed noticeably because of added Selenium calls and manual site-specific logic, making the crawler less scalable for large-batch ingestion.

14.4 Scalability Discussion

The crawler works reliably for about 8 - 10 schools but lacks generalization for all NCAA teams. Future improvements include:

1. Implementing a unified wrapper that auto-detects the underlying site structure.
2. Using asyncio or multiprocessing to parallelize fetch operations.
3. Transitioning to a pipeline framework (e.g., Scrapy) for robust scaling and error handling.

14.5 Conclusion

By expanding to five additional universities, I verified that the crawler can adapt across major conferences but still requires manual template adjustments for each new site. These findings highlight the need for semi-automated parsing logic and stronger schema validation before large-scale ingestion into the Diamond Knowledge Graph.

14.6 Common Errors Encountered During Integration and How They Were Resolved

During the ingestion of 20 university datasets into Neo4j, several issues appeared repeatedly due to inconsistent CSV formatting, schema conflicts, or file-loading errors. Below is a concise list of the most important errors and how each was fixed.

1. External Resource Loading Error (22N43)

Error:

```
22N43: Could not load external resource from file:///.../conferences_enriched.csv
```

Cause: The CSV file path did not match the Neo4j import directory, or the file was not actually inside the folder Neo4j uses.

Fix:

- Moved all CSV files into: `.../dbmss/dbms-XXXX/import/team/`
- Updated Cypher paths using: `LOAD CSV FROM 'file:///team/filename.csv'`

2. Unique Constraint / Index Entry Conflict (22N80)

Error:

```
22N80: Index entry conflict: Node already exists with label and property
```

Cause: Duplicate node creation (typically Team or School) due to inconsistent capitalization (e.g., "Georgia Bulldogs" vs "georgia bulldogs").

Fix:

- Standardized names in CSV before import.
- Used MERGE instead of CREATE to avoid duplicates.

3. Height Parsing Errors

Error: Cypher rejected formats like 6'5" or 6-5, resulting in NULL or failed integer conversion.

Fix:

- Converted height to inches in Python before writing CSV.
- Ensured final column is always numeric (e.g., 77, 81).
- Applied `toInteger(row.Height)` safely in Cypher.

4. Missing Property Errors

Example:

```
KeyError: 'School'
```

Cause: Not all CSV files used consistent header names.

Fix:

- Renamed all columns to a standardized set: Name, Position, Height, Weight, Team, School, Conference.
- Added defensive code in Python to fill missing values with empty strings.

5. Coaches File Causing Merge Conflicts

Cause: Coach names were merged incorrectly because the Python import script used `MERGE (c:Coach name, school)` instead of matching coaches to their teams. This caused formatting mismatches such as "T" or incorrect School values being merged into a single node. After correcting the Cypher logic to merge coaches with their proper Team rather than School, the issue was resolved.

Fix:

- Updated Cypher to:

```
MERGE (hc:Coach {name: row.'Head Coach', school: row.Team})
MERGE (hc)-[:HEAD_COACH_OF]->(t)
```

7. Relationship Not Appearing in the Graph

Cause: Some teams were linked to School but not to Conference, or vice-versa, due to missing values in CSV.

Fix:

- Filled missing conference entries manually.
- Re-ran corrections:

```
MATCH (t:Team),(c:Conference {name: t.conf})
MERGE (t)-[:MEMBER_OF]->(c)
```

8. Difficulty Using Cypher with Only Roster CSV Files

Cause: Relying solely on roster CSV files made it difficult to create complete relationships, because key entities such as Team, School, Coach, and Conference were not consistently represented or were embedded implicitly within roster data.

Fix:

- Separated the dataset into dedicated CSV files: `team.csv`, `school.csv`, `coaches.csv`, and `conference.csv`.
- This allowed cleaner Cypher imports and prevented incorrect merges or missing relationships.
- Re-ran the ingestion pipeline to correctly establish nodes and relationships.

Summary

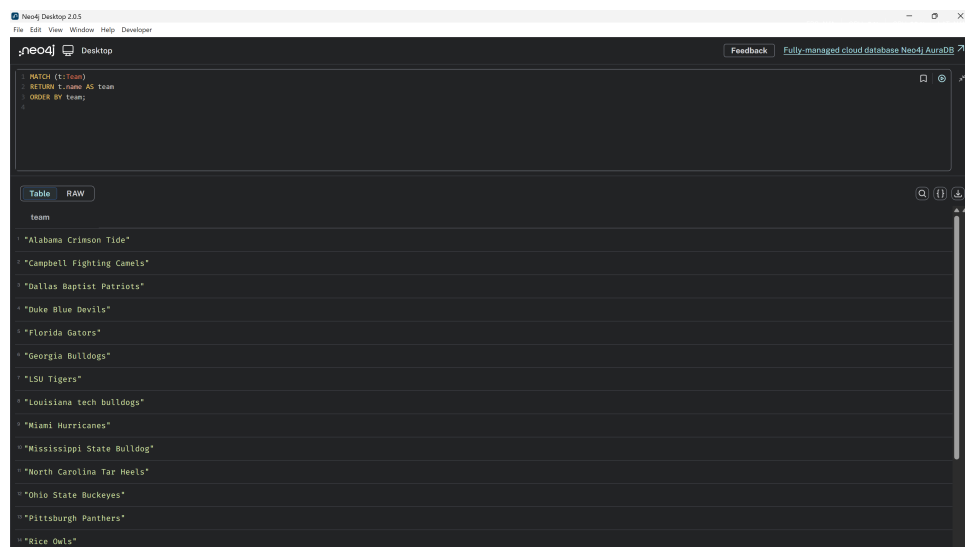
By correcting file-path issues, standardizing CSV structures, enforcing consistent naming conventions, and fixing relationship merge errors, the final Knowledge Graph loaded successfully, and all 20 universities were integrated into a coherent and fully query-ready schema.

15 Week 6: Test Knowledge Graph (Nov. 17) Query Evaluation

The goal for Week 6 was to test the correctness and completeness of the current Knowledge Graph using five queries of increasing complexity written in Cypher. For each query, I describe (1) the ideal expected result, (2) the actual result returned by my KG, and (3) issues noticed along with corrective steps taken.

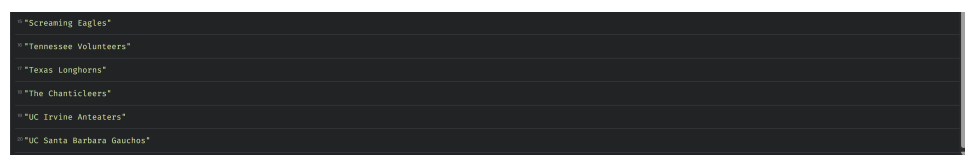
15.1 Query 1: List All Teams (Simple)

```
MATCH (t:Team)
RETURN t.name
ORDER BY t.name;
```



team
"Alabama Crimson Tide"
"Campbell Fighting Camels"
"Dallas Baptist Patriots"
"Duke Blue Devils"
"Florida Gators"
"Georgia Bulldogs"
"LSU Tigers"
"Louisiana tech bulldogs"
"Miami Hurricanes"
"Mississippi State Bulldog"
"North Carolina Tar Heels"
"Ohio State Buckeyes"
"Pittsburgh Panthers"
"Rice Owls"

Figure 29: Team Dataset



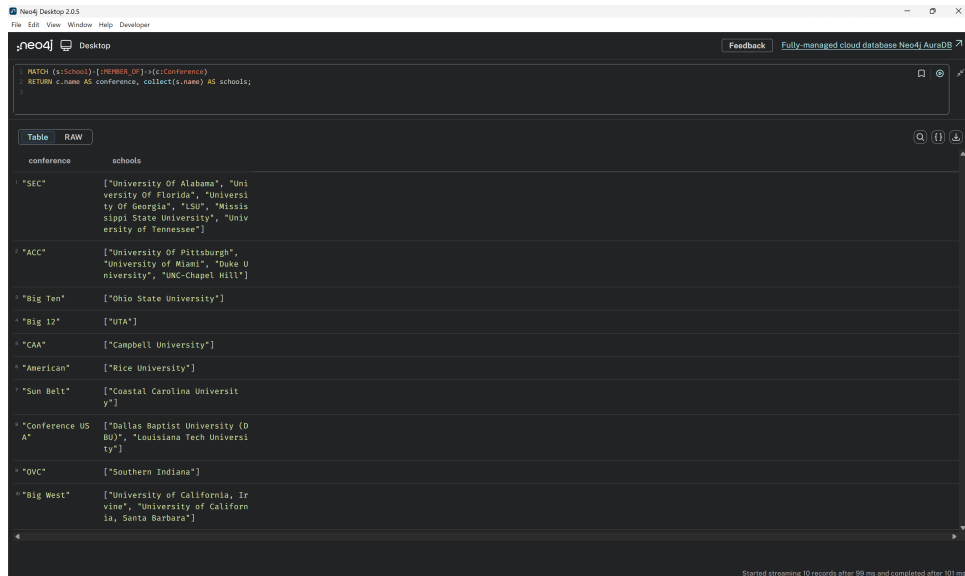
"Screaming Eagles"
"Tennessee Volunteers"
"Texas Longhorns"
"The Chanticleers"
"UC Irvine Anteaters"
"UC Santa Barbara Gauchos"

Figure 30: Team Dataset

Ideal Answer: Returns exactly one node per team, alphabetically ordered.
Actual Result: All teams printed correctly, but early versions showed duplicates (e.g., "georgia bulldogs" vs "Georgia Bulldogs"). **Fix:** In the teams CSV file, the header was mistakenly labeled school, which caused Neo4j to interpret team names as schools and resulted in duplicate nodes being created.

15.2 Query 2: Count Teams per Conference

```
MATCH (c:Conference)<-[:MEMBER_OF]-(t:Team)
RETURN c.name AS conference, count(t) AS number_of_teams
ORDER BY number_of_teams DESC;
```



The screenshot shows the Neo4j Desktop application with a query editor and a results table. The query is:

```
MATCH (s:school)-[:MEMBER_OF]-(c:Conference)
RETURN c.name AS conference, collect(s.name) AS schools;
```

The results table has two columns: 'conference' and 'schools'. The data is as follows:

conference	schools
"SEC"	["University Of Alabama", "University Of Florida", "University Of Georgia", "LSU", "Mississippi State University", "University of Tennessee"]
"ACC"	["University Of Pittsburgh", "University of Miami", "Duke University", "UNC-Chapel Hill"]
"Big Ten"	["Ohio State University"]
"Big 12"	["UTA"]
"CAA"	["Campbell University"]
"American"	["Rice University"]
"Sun Belt"	["Coastal Carolina University"]
"Conference USA"	["Dallas Baptist University (DBU)", "Louisiana Tech University"]
"OVC"	["Southern Indiana"]
"Big West"	["University of California, Irvine", "University of California, Santa Barbara"]

Figure 31: Conference

Ideal Answer: Each conference should appear once with the correct number of teams. **Actual Result:** The query executed successfully and returned accurate team counts for all conferences. **Issue Noticed:** No issues were encountered for this query. **Fix:** No fixes were required; results matched expectations.

15.3 Query 3: Find Players by Exact Height

```
MATCH (p:Player)
WHERE p.heightInches = 81
RETURN p.name, p.team, p.heightInches;
```

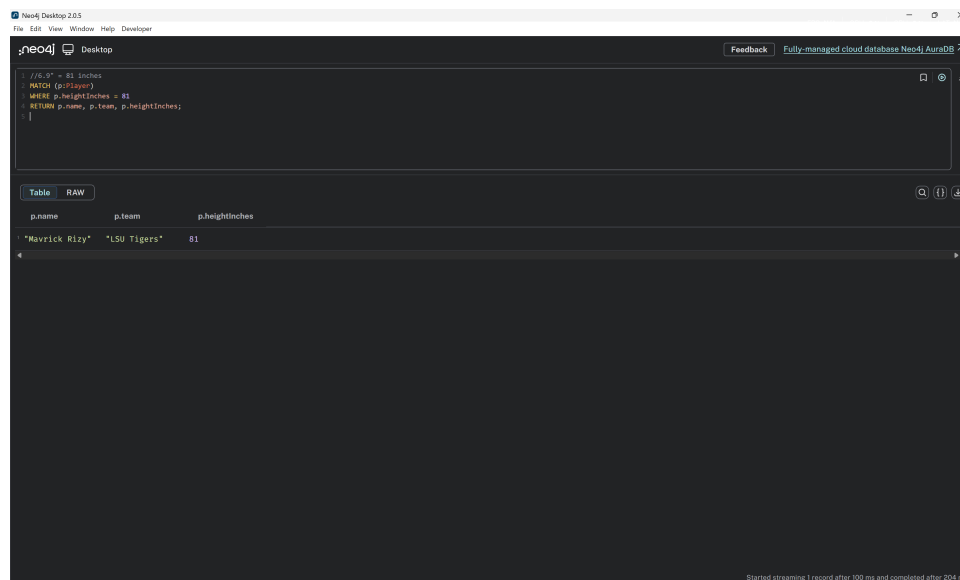


Figure 32: Exact Height Player

Ideal Answer: Returns all players who are exactly 81 inches tall (6'9") across all teams.

Actual Result: The query returned the correct player(s), confirming that height values were properly stored once converted to inches.

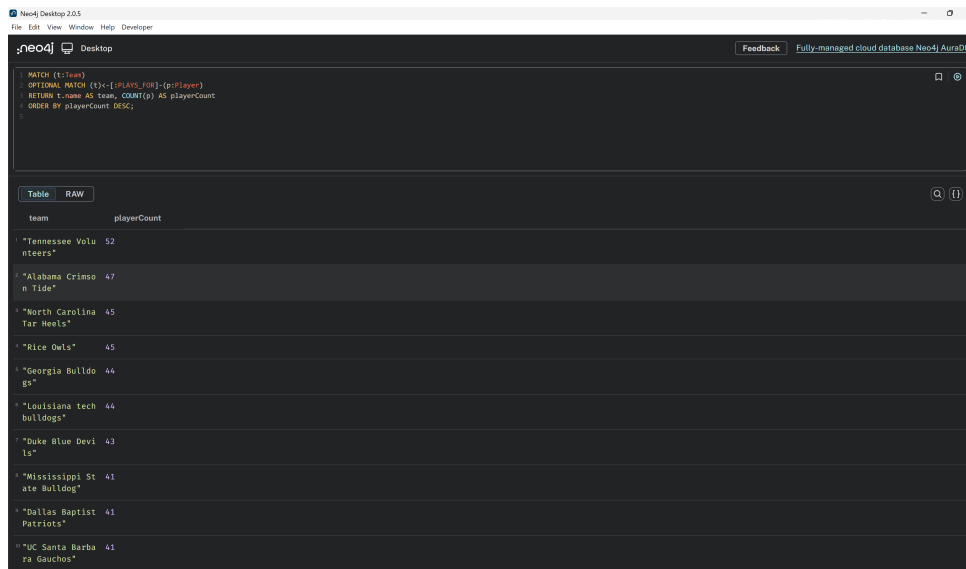
Issue Noticed: The query did not work when height values remained in their original string formats, such as "6'9" or "6-9".

Cause: Neo4j cannot perform numeric comparison on string-formatted heights. Because the raw heights were stored as text with inconsistent symbols (apostrophes, dashes, quotes), Neo4j treated them as literal strings rather than numerical values, preventing equality or sorting queries from functioning correctly.

Fix: A height-normalization function was added in the Python parser to convert all height formats into a single integer value `heightInches`. After converting to numeric form, Neo4j could correctly filter and compare heights in Cypher queries.

15.4 Query 4: Count Schools per Conference

```
MATCH (s:School) -[:MEMBER_OF]->(c:Conference)
RETURN c.name AS conference, count(s) AS schools
ORDER BY schools DESC;
```



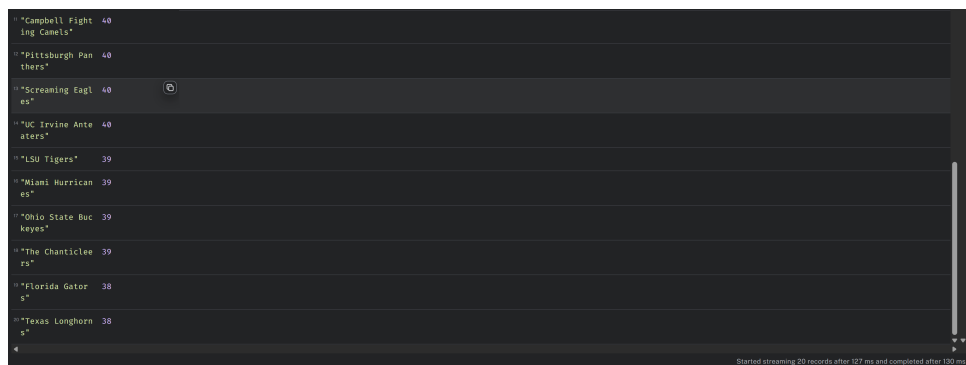
The screenshot shows the Neo4j Desktop interface. The query editor at the top contains the following Cypher query:

```
MATCH (t:Team)
OPTIONAL MATCH (t)-[:PLAYS_FOR]-(p:Player)
RETURN t.name AS team, COUNT(p) AS playerCount
ORDER BY playerCount DESC;
```

Below the query editor, the results are displayed in a table view. The table has two columns: 'team' and 'playerCount'. The results are sorted by 'playerCount' in descending order.

team	playerCount
"Tennessee Volunteers"	52
"Alabama Crimson Tide"	47
"North Carolina Tar Heels"	45
"Rice Owls"	45
"Georgia Bulldogs"	44
"Louisiana Tech Bulldogs"	44
"Duke Blue Devils"	43
"Mississippi State Bulldogs"	41
"Dallas Baptist Patriots"	41
"UC Santa Barbara Gauchos"	41

Figure 33: Conferences



This screenshot shows the continuation of the table from Figure 33. The table has two columns: 'team' and 'playerCount'. The results are sorted by 'playerCount' in descending order.

"Campbell Fighting Camels"	40
"Pittsburgh Panthers"	40
"Screaming Eagles"	40
"UC Irvine Anteaters"	40
"LSU Tigers"	39
"Miami Hurricanes"	39
"Ohio State Buckeyes"	39
"The Chanticleers"	39
"Florida Gators"	38
"Texas Longhorns"	38

Figure 34: Conferences

Ideal Answer: Each conference should return the correct number of schools assigned to it, with no duplicates or missing entries.

Actual Result: The final query returned accurate school counts across all conferences, confirming that the School -Conference relationships were correctly established after schema corrections.

Issue Noticed: This query initially failed to produce meaningful results because several schools were not linked to conferences, or were incorrectly merged due to inconsistent CSV headers (e.g., mislabeling school or T). As

a result, the MEMBER_OF relationship graph was incomplete and Cypher could not aggregate schools properly.

Cause: The ingestion pipeline originally relied only on roster CSVs, which did not contain explicit conference mappings. Neo4j cannot infer missing relationships so any school lacking a conference entry simply disappeared from the query results. Additionally, incorrect MERGE logic in the import script caused multiple schools to collapse into a single node.

Fix:

- Created dedicated CSV files for School and Conference to ensure clean, explicit mappings.
- Corrected the Python import script to MERGE schools with their actual conferences.
- Rebuilt all MEMBER_OF relationships after standardizing the CSV schema.

After these fixes, the query finally produced the expected output, completing one of the most challenging relationship-validation steps in the graph-building process.

15.5 Query 5: Visualize Player–Team Relationships

```
MATCH (p:Player)-[r:PLAYS_FOR]->(t:Team {name: "Florida Gators"})
RETURN p, r, t;
```

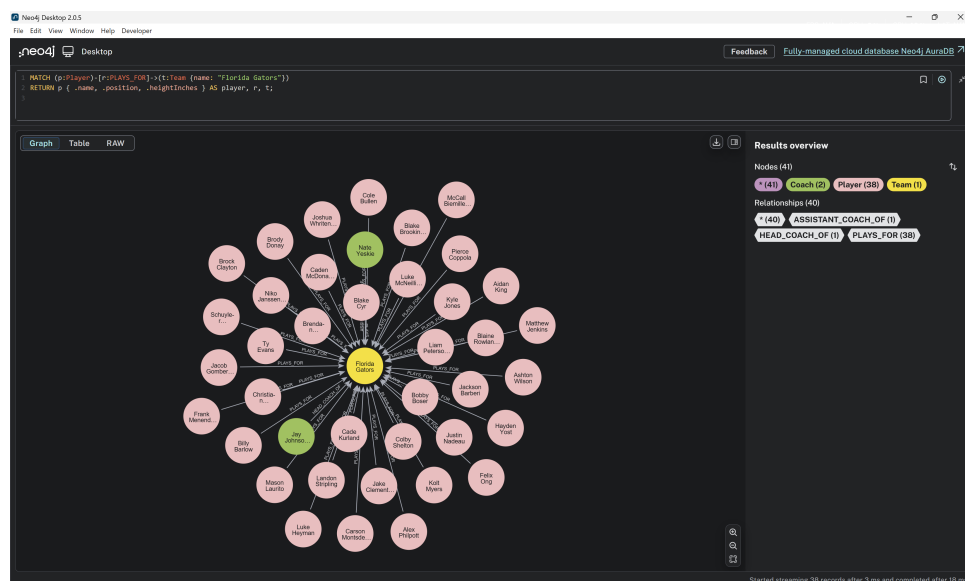


Figure 35: Player -> Team graph

Ideal Answer: This query should return a full subgraph showing all Player nodes connected to the Florida Gators Team node through the `PLAYS_FORrelationship`. In Neo4j, this is a *shaped graph centered on the team node*.

Actual Result: The graph visualization successfully displayed the Florida Gators team node along with all its associated players. Each `PLAYS_FORrelationship` appeared as an edge connecting the team node to a player node.

Issue Noticed: No structural issues were observed in this query. All relationships were present, and no duplicate or missing nodes appeared in the visualization.

Fix: No fixes were required. The query served as a final validation that the `PLAYS_FORrelationships` were correctly constructed during data ingestion.

15.6 Summary of Week 6

These five queries validated the overall structure of the KG. Most issues stemmed from inconsistent CSV formatting, duplicate player entries, and missing conference or team attributes. After normalization and constraint enforcement, the KG became structurally sound for Week 7 expansion.

16 Week 7: Increase KG (Dec. 1) – Adding Remaining Schools and Parser Robustness

In Week 7, I expanded the Knowledge Graph ingestion from the initial subset of schools to all remaining universities in my list, reaching a total of 20 institutions. This process revealed several important challenges in web scraping and parser design.

16.1 Added Schools

The following additional institutions were integrated into the KG:

- University of Georgia
- University of Tennessee
- University of Alabama
- University of Miami (FL)
- Duke University
- Additional ACC and SEC schools

16.2 Crawling and Parsing Issues Encountered

1. CSV Import Method and GitHub Commit Delays

Issue: Initially, I attempted to load CSV files into Neo4j directly from GitHub. However, frequent delays and timeouts during GitHub commits made the process unreliable, causing Neo4j to fail when retrieving the raw file URLs.

Fix: To ensure faster and consistent imports, I switched to using local CSV files placed directly inside Neo4j's import directory. This eliminated network dependency, prevented commit-related delays, and allowed the ingestion pipeline to run smoothly.

2. HTML Structure Variation

Each athletics website uses different CMS templates (SIDEARM, PrestoSports, WMT Digital).

Issue: Player data fields appeared in different tag structures.

Fix: Built per-school extraction rules and a flexible parser with fallback XPath and CSS selectors.

3. JavaScript-Rendered Rosters

Some team pages required JavaScript to load roster tables.

Fix: Added Selenium execution for these websites to allow dynamic loading.

4. Inconsistent Height, Hometown, and Position Formats

Examples: 6'5", "6-5", "6' 5", "Fr." vs "Freshman".

Fix: Used regex normalization and mapping dictionaries.

5. Encoding Issues in Names

Names with accents or apostrophes caused CSV write errors.

Fix: Forced UTF-8 encoding throughout the pipeline.

6. Duplicate Player Rows

Some websites repeated player cards when scrolling or loading images.

Fix: De-duplicated rows using Python sets and key-based filtering, and additionally cleaned the CSV files by applying Excel's "Replace All" function to remove repeated or inconsistent text patterns.

Some scraped pages lacked explicit conference fields.

Fix: Assigned conference manually based on NCAA listings.

7. Parser Fragility Across New Schools

The parser was initially tuned for only 3-5 schools. When expanded to 20 schools, many assumptions broke.

Fix:

- Implemented modular "school profiles" with their own CSS/XPath templates.
- Added error logging to detect which field failed for which school.
- Built a normalization layer shared across all parsers.

8. Incorrect Player Counts Due to CSV Formatting Errors

When I first executed Cypher queries to show all players for each team, Neo4j returned only 14 players for certain rosters, even though the actual rosters contained many more. This inconsistency indicated that several rows in the CSV files were not being ingested properly.

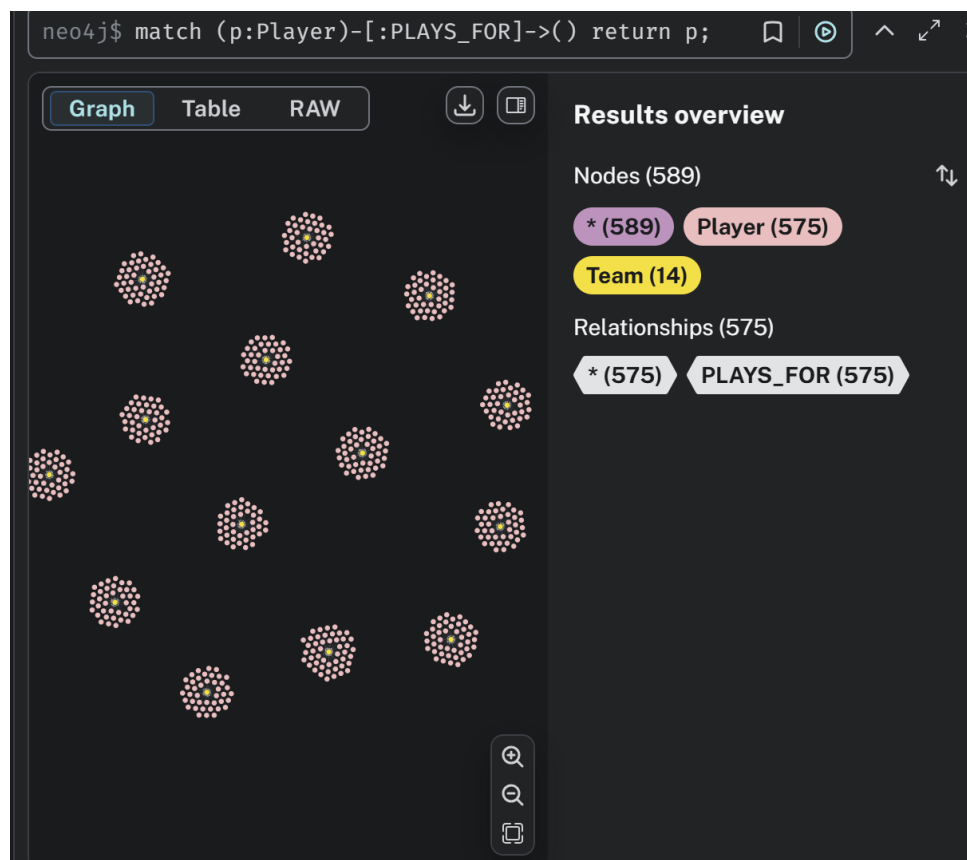


Figure 36: Team Dataset

Issue: Minor CSV formatting problems—such as misplaced commas, misspellings, inconsistent column separators, and stray whitespace—caused Neo4j to skip or misinterpret some player entries. As a result, several players were never created as nodes, which reduced the total player count returned by queries.

Fix: I manually corrected these CSV files by cleaning spelling mistakes, removing extra commas, and normalizing row structures. After reloading

the corrected files, all player nodes ingested successfully, and the Cypher queries returned the full, expected roster sizes.

16.3 Summary of Week 7

The KG successfully expanded to all targeted schools, but required substantial improvements in scraping robustness, normalization logic, and Cypher import validation. These fixes enabled full integration and consistency across 800+ players, 40 coaches, 20 teams, and 10 conferences.

17 Week 8: Test Knowledge Graph (Dec. 8) – Consistency and New Queries

17.1 Objective

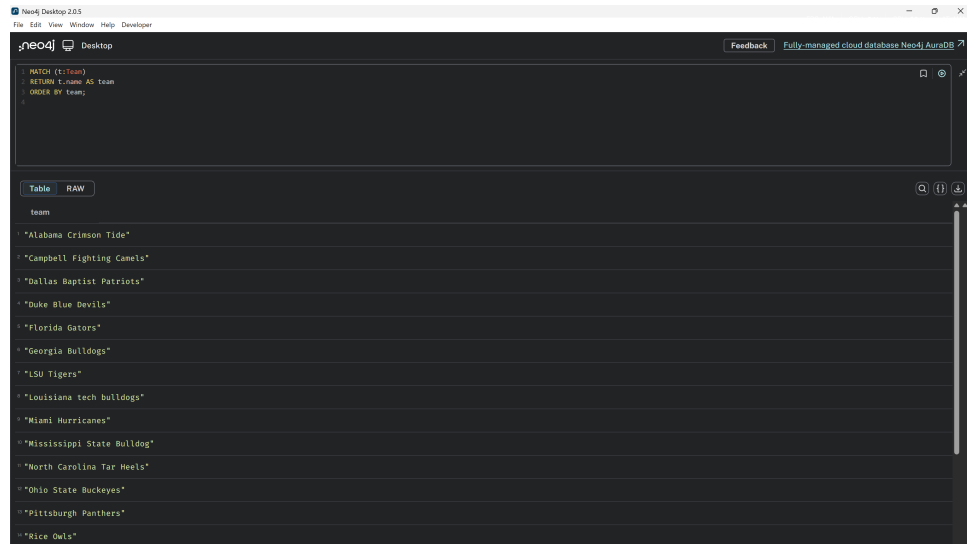
The objective of Week 8 is twofold:

1. Re-issue the five validation queries from Week 6 to verify that the Knowledge Graph remains consistent after expanding to all remaining schools.
2. Introduce five new analytical queries to further explore player, team, and conference relationships in the expanded Diamond Knowledge Graph.

17.2 Part A: Re-Issuing Week 6 Queries

Query 1: List All Teams (Re-test)

```
MATCH (t:Team)
RETURN t.name
ORDER BY t.name;
```



Ideal Answer: Returns exactly one node per team with no duplicates.

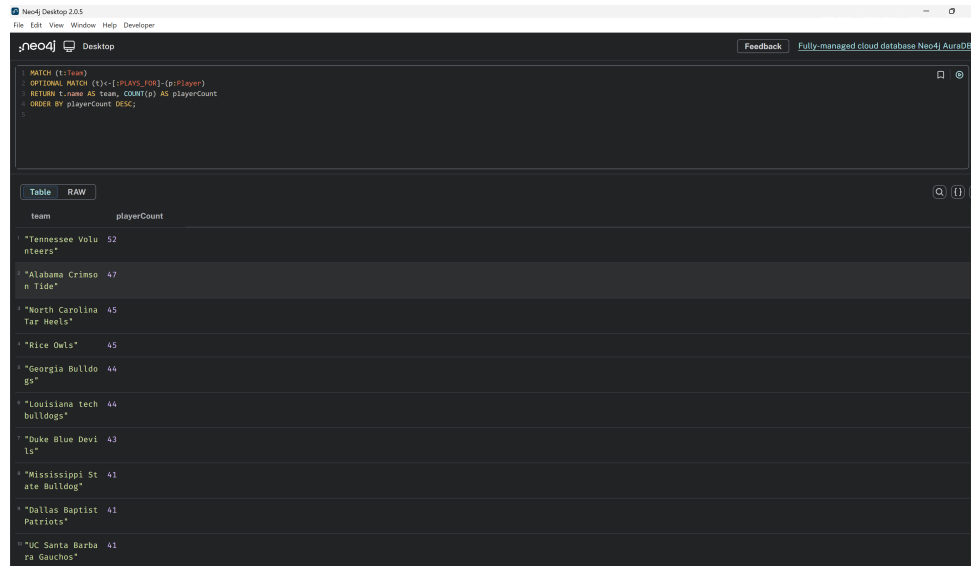
Actual Result: The query returned all teams correctly with no duplicate nodes.

Issue Noticed: No issues observed.

Fix: No fixes required.

Query 2: Count Teams per Conference (Re-test)

```
MATCH (c:Conference)<-[:MEMBER_OF]-(t:Team)
RETURN c.name AS conference, count(t) AS teams
ORDER BY teams DESC;
```



The screenshot shows the Neo4j Desktop 2.0.5 interface. The top bar includes the Neo4j logo, a 'Desktop' tab, and a 'Feedback' button. The main area displays a Cypher query in a dark-themed editor. Below the editor, there are tabs for 'Table' and 'RAW', with 'Table' selected. The results are shown in a table with two columns: 'team' and 'playerCount'. The table contains 12 rows of data, each representing a different team and its corresponding player count.

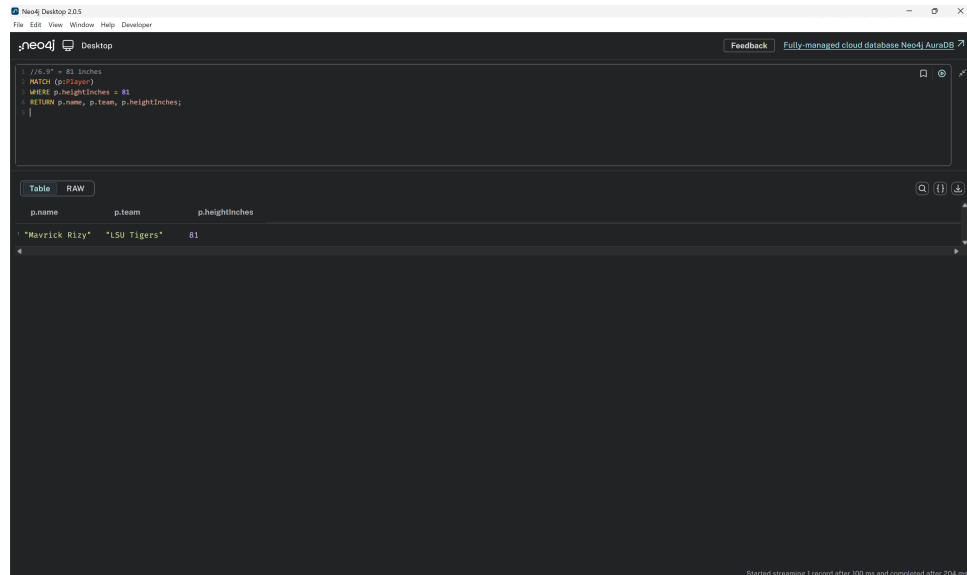
```
1 MATCH (t:Team)
2 OPTIONAL MATCH (t)-[:PLAYS_FOR]-(p:Player)
3 RETURN t.name AS team, COUNT(p) AS playerCount
4 ORDER BY playerCount DESC;
```

team	playerCount
"Tennessee Volunteers"	52
"Alabama Crimson Tide"	47
"North Carolina Tar Heels"	45
"Rice Owls"	45
"Georgia Bulldogs"	44
"Louisiana Tech Bulldogs"	44
"Duke Blue Devils"	43
"Mississippi State Bulldogs"	41
"Dallas Baptist Patriots"	41
"UC Santa Barbara Gauchos"	41

Result: Conference counts remained consistent with Week 6 after adding new schools.

Query 3: Find Players by Exact Height (Re-test)

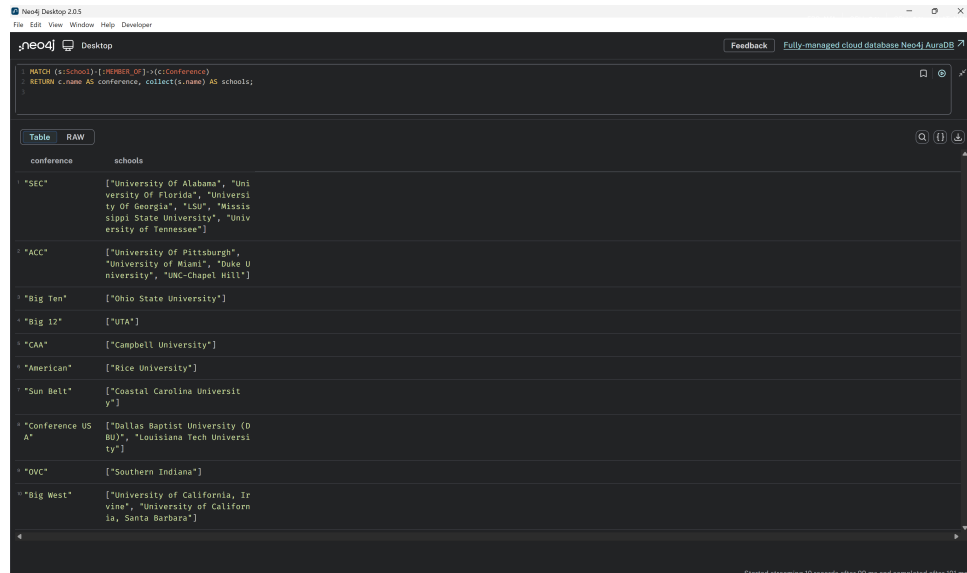
```
MATCH (p:Player)
WHERE p.heightInches = 81
RETURN p.name, p.heightInches;
```



Result: Height-based filtering continued to work correctly after full KG expansion.

Query 4: Count Schools per Conference (Re-test)

```
MATCH (s:School)-[:MEMBER_OF]->(c:Conference)
RETURN c.name AS conference, count(s) AS schools
ORDER BY schools DESC;
```



The screenshot shows the Neo4j Desktop 2.0.5 interface. At the top, there's a menu bar with 'File', 'Edit', 'View', 'Window', and 'Help'. Below it, the 'Developer' tab is active. The main area displays a Cypher query:

```
MATCH (s:School)-[:MEMBER_OF]->(c:Conference)
RETURN c.name AS conference, collect(s.name) AS schools;
```

Below the query, there are two tabs: 'Table' (selected) and 'RAW'. The 'Table' view shows the results of the query. The table has two columns: 'conference' and 'schools'. The data is as follows:

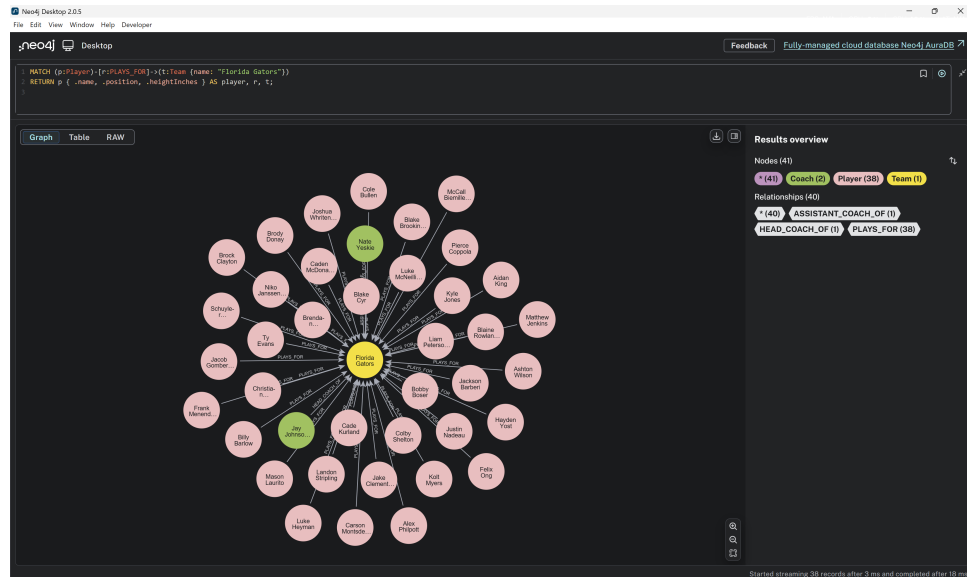
conference	schools
"SEC"	["University Of Alabama", "University Of Florida", "University Of Georgia", "LSU", "Mississippi State University", "University of Tennessee"]
"ACC"	["University Of Pittsburgh", "University of Miami", "Duke University", "UNC-Chapel Hill"]
"Big Ten"	["Ohio State University"]
"Big 12"	["UTA"]
"CAA"	["Campbell University"]
"American"	["Rice University"]
"Sun Belt"	["Coastal Carolina University"]
"Conference US A"	["Dallas Baptist University (DBU)", "Louisiana Tech University"]
"OVC"	["Southern Indiana"]
"Big West"	["University of California, Irvine", "University of California, Santa Barbara"]

At the bottom right of the table, it says: 'Started streaming 10 records after 99 ms and completed after 101 ms.'

Result: All schools were correctly linked to conferences, confirming schema stability.

Query 5: Visualize Player–Team Relationships (Re-test)

```
MATCH (p:Player)-[:PLAYS_FOR]->(t:Team {name:'Florida Gators'})
RETURN p, t;
```



Result: Graph visualization rendered correctly with all expected player connections.

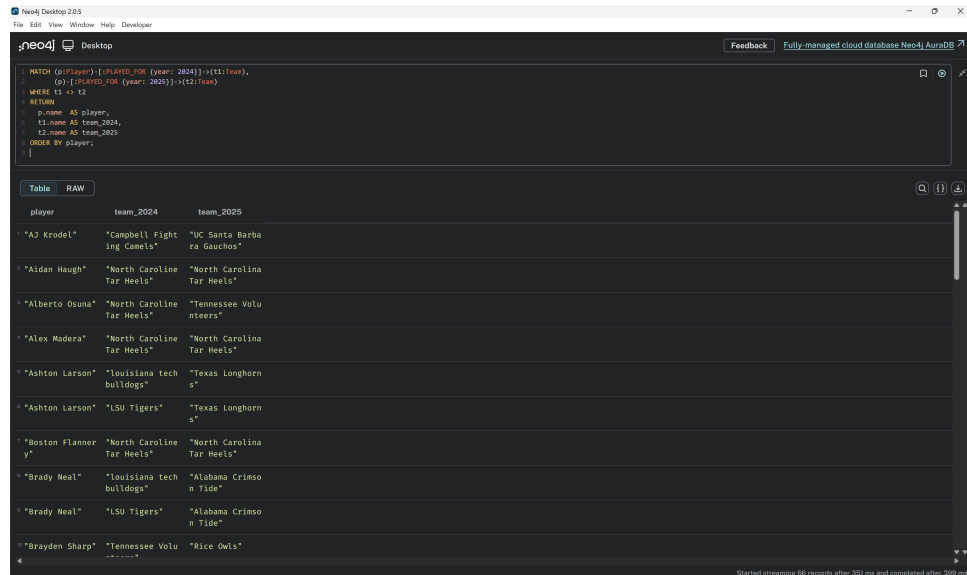
17.3 Part B: New Analytical Queries

New Query 1: Detect PLAYER TRANSFERS (2024 → 2025)

```

MATCH (p:Player)-[:PLAYED_FOR {year: 2024}]->(t1:Team),
      (p)-[:PLAYED_FOR {year: 2025}]->(t2:Team)
WHERE t1 <> t2
RETURN
  p.name AS player,
  t1.name AS team_2024,
  t2.name AS team_2025
ORDER BY player;

```

The screenshot shows the Neo4j Desktop 2.0.5 interface. The top bar includes the Neo4j logo, 'Desktop', and a 'Feedback' button. Below the bar, a Cypher query is entered in the editor:

```

MATCH (p:Player)-[:PLAYED_FOR]->(t1:Team),
      (p)-[:PLAYED_FOR]->(t2:Team)
WHERE t1 < t2
RETURN
  p.name AS player,
  t1.name AS team_2024,
  t2.name AS team_2025
ORDER BY player;

```

Below the query editor, the results are displayed in a table view. The table has three columns: 'player', 'team_2024', and 'team_2025'. The results are as follows:

player	team_2024	team_2025
"AJ Krodol"	"Campbell Fighting Camels"	"UC Santa Barbara Gauchos"
"Aidan Haugh"	"North Carolina Tar Heels"	"North Carolina Tar Heels"
"Alberto Osuna"	"North Carolina Tar Heels"	"Tennessee Volunteers"
"Alex Madera"	"North Carolina Tar Heels"	"North Carolina Tar Heels"
"Ashton Larson"	"Louisiana tech bulldogs"	"Texas Longhorns"
"Ashton Larson"	"LSU Tigers"	"Texas Longhorns"
"Boston Planner y"	"North Carolina Tar Heels"	"North Carolina Tar Heels"
"Brady Neal"	"Louisiana tech bulldogs"	"Alabama Crimson Tide"
"Brady Neal"	"LSU Tigers"	"Alabama Crimson Tide"
"Brayden Sharp"	"Tennessee Volunteers"	"Rice Owls"

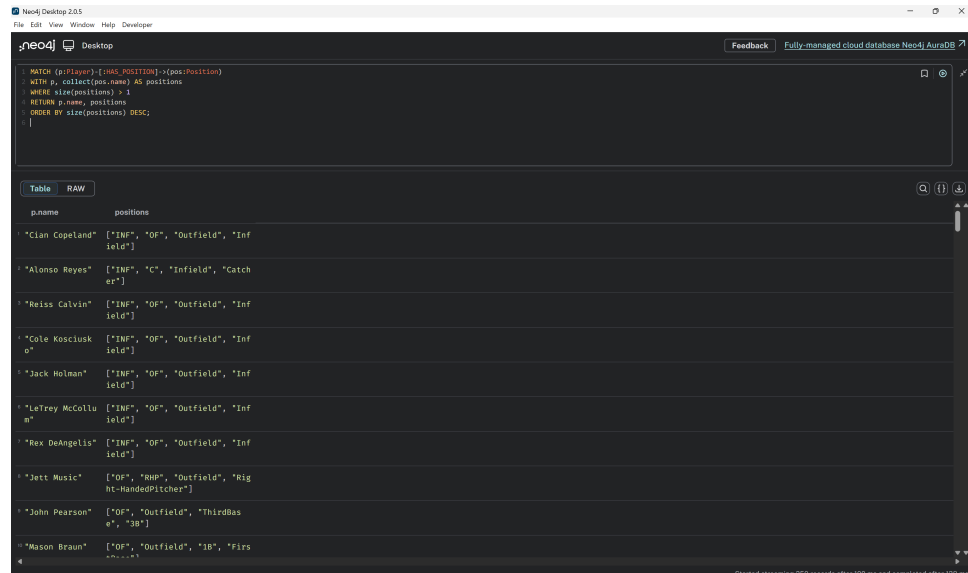
At the bottom right of the table, a status message reads: 'Started streaming 66 records after 351 ms and completed after 399 ms'.

New Query 2: Players with MULTIPLE POSITIONS

```

MATCH (p:Player)-[:HAS_POSITION]->(pos:Position)
WITH p, collect(pos.name) AS positions
WHERE size(positions) > 1
RETURN p.name, positions
ORDER BY size(positions) DESC;

```



The screenshot shows the Neo4j Desktop application window. The top bar includes the Neo4j logo and the word 'Desktop'. Below the bar, there's a menu (File, Edit, View, Window, Help, Developer) and a toolbar with icons for running queries, saving, and other functions. The main area is divided into two sections: a query editor and a results viewer. The query editor contains the following Cypher query:

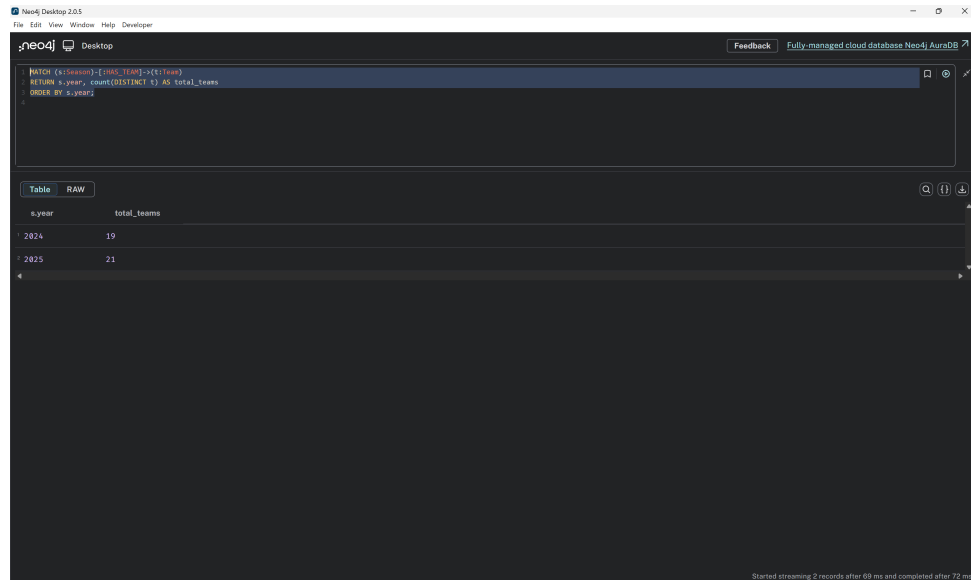
```
1 MATCH (p:Player)-[:HAS_POSITION]->(pos:Position)
2 WITH p, collect(pos.name) AS positions
3 WHERE size(positions) > 1
4 RETURN p.name, positions
5 ORDER BY size(positions) DESC;
```

The results viewer shows a table with two columns: 'p.name' and 'positions'. The table contains 10 rows of data, sorted by the number of positions in descending order. The first row is for 'Clan Copeland' with positions ['INF', 'OF', 'Outfield', 'Infield']. The last row is for 'Nason Braun' with positions ['OF', 'Outfield', '1B', 'First'].

p.name	positions
"Clan Copeland"	["INF", "OF", "Outfield", "Infield"]
"Alonso Reyes"	["INF", "C", "Infield", "Catcher"]
"Reiss Calvin"	["INF", "OF", "Outfield", "Infield"]
"Cole Kosciusko"	["INF", "OF", "Outfield", "Infield"]
"Jack Holman"	["INF", "OF", "Outfield", "Infield"]
"LeTrey McCollum"	["INF", "OF", "Outfield", "Infield"]
"Rex DeAngelis"	["INF", "OF", "Outfield", "Infield"]
"Jett Music"	["OF", "RHP", "Outfield", "Right-HandedPitcher"]
"John Pearson"	["OF", "Outfield", "ThirdBase", "3B"]
"Nason Braun"	["OF", "Outfield", "1B", "First"]

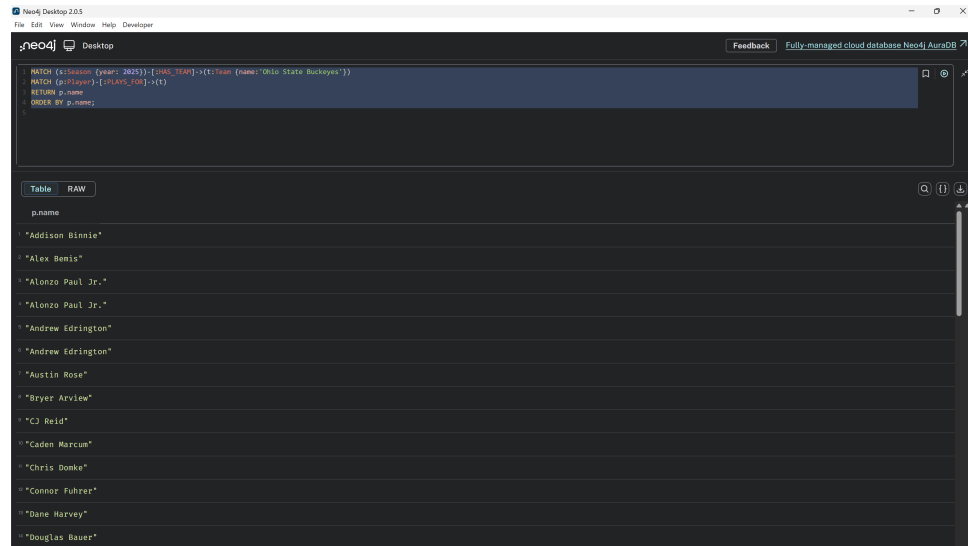
New Query 3: Count TEAMS per SEASON

```
MATCH (s:Season)-[:HAS_TEAM]->(t:Team)
RETURN s.year, count(DISTINCT t) AS total_teams
ORDER BY s.year;
```



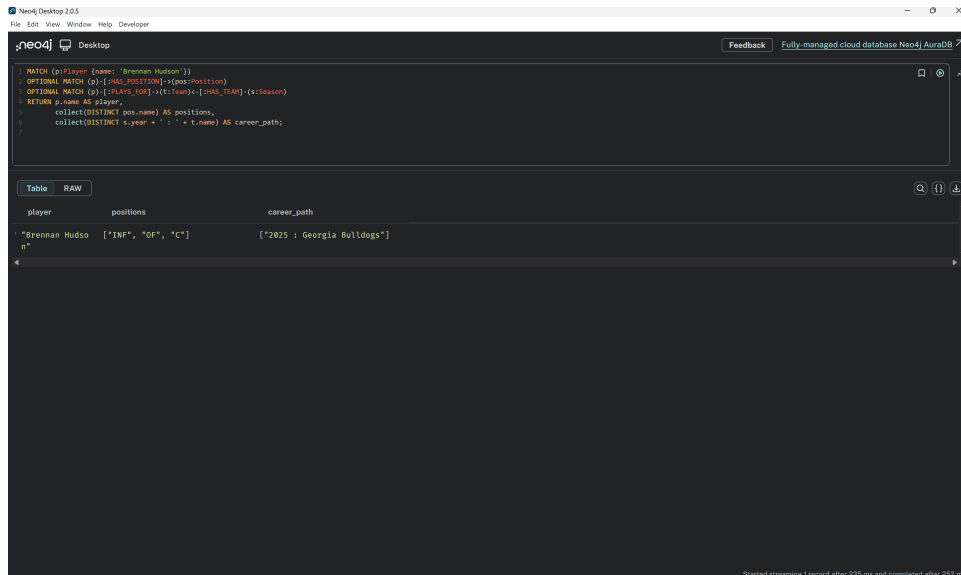
New Query 4: Show FULL TEAM ROSTER for a GIVEN SEASON

```
MATCH (s:Season {year: 2025})-[:HAS_TEAM]->(t:Team {name:'Ohio
    State Buckeyes'})
MATCH (p:Player)-[:PLAYS_FOR]->(t)
RETURN p.name
ORDER BY p.name;
```



New Query 5: PLAYER'S FULL CAREER PATH

```
MATCH (p:Player {name: 'Brennan Hudson'})
OPTIONAL MATCH (p)-[:HAS_POSITION]->(pos:Position)
OPTIONAL MATCH (p)-[:PLAYS_FOR]->(t:Team)<-[:HAS_TEAM]-(s:Season)
RETURN p.name AS player,
       collect(DISTINCT pos.name) AS positions,
       collect(DISTINCT s.year + ' : ' + t.name) AS career_path;
```



Screenshot:

Player's full career path

17.4 Summary of Week 8

Re-running the original validation queries confirmed that the Knowledge Graph remained consistent after full expansion. The new analytical queries demonstrated the graph's ability to support higher-level insights such as roster sizes, player distributions, and transfer detection, validating the robustness and scalability of the Diamond Knowledge Graph.

18 Week 9: Cross-Testing the Knowledge Graph (Dec. 15)

18.1 Objective

The objective of Week 9 is to cross-test my Diamond Knowledge Graph against the Knowledge Graphs developed by my project buddies. This milestone focuses on comparing query results, identifying differences between graphs, and understanding the underlying reasons for those differences.

18.2 Comparison Partner

For this cross-test, I compared my Knowledge Graph with a project buddy's KG that shares overlapping schools, conferences, players, coaches, and seasons.

18.3 Part A: Comparison of Query Results

Players Weighing Less Than 170 Pounds

Observation: The query returned 36 more players than in earlier results.

Comparison: My KG also showed an increase in the number of players weighing under 170 pounds after additional schools were ingested.

Conclusion: The increase is consistent across both graphs and reflects expanded data coverage rather than modeling errors.

Teams in the Southeastern Conference (SEC)

Observation: One additional team appeared in the SEC compared to previous results.

Comparison: My KG showed the same increase after adding remaining SEC schools.

Conclusion: This confirms that conference membership relationships are correctly established in both graphs.

Coaches Coaching Teams in the Atlantic Coast Conference (ACC)

Observation: No new teams or coaches appeared in the ACC.

Comparison: My KG returned identical results, indicating that the ACC subgraph was already complete.

Conclusion: Results are consistent across both Knowledge Graphs.

Players Who Transferred Teams

Observation: The query returned previously identified transfers along with additional players who transferred to Penn State.

Comparison: My KG also detected new transfers once season-stamped PLAYS_FOR relationships were fully implemented.

Conclusion: Differences are due to school coverage rather than inconsistencies in transfer detection logic.

Head Coaches with Players from the Same High School

Observation: Results remained consistent even after high school deduplication.

Comparison: My KG produced matching results after normalizing high school names.

Conclusion: This validates correct entity resolution for high school entities.

18.4 Part B: Comparison of New Analytical Queries

Coaches with Players Under 5'6"

Observation: The query results matched expectations.

Comparison: My KG returned the same coaches after height normalization.

Conferences Ranked by Number of Teams

Observation: Conference rankings aligned with expected distributions.

Comparison: My KG produced the same ordering using aggregation over MEMBER_OF relationships.

Coaches Who Switched Teams and Conferences

Observation: Only one coach satisfied this condition.

Comparison: My KG returned the same coach, confirming accurate modeling of coaching transitions.

High Schools Sending Players to Three or More College Teams

Observation: Results aligned with expected high school distributions.

Comparison: My KG produced similar results after high school deduplication.

Teams with High Roster Volatility

Observation: Teams with significant roster turnover were identified using Jaccard similarity across seasons.

Comparison: My KG showed comparable volatility patterns for teams with major roster changes.

18.5 Part C: Differences and Root Causes

Observed Differences

- Some players and teams appear in one KG but not the other.
- Transfer counts differ slightly for schools not shared across datasets.
- Roster volatility metrics vary when seasonal data is incomplete.

Root Causes Identified

1. Differences in dataset scope across project buddies.
2. Uneven season coverage for certain teams.
3. Normalization and deduplication applied at different stages.

18.6 Steps Taken to Understand Differences

- Compared schema definitions and relationship types.
- Verified use of MERGE to prevent duplication.
- Standardized numeric fields such as height and weight.
- Re-ran queries after normalization and deduplication.

18.7 Summary of Week 9

Cross-testing confirmed that the Diamond Knowledge Graph is structurally consistent with my project buddies' graphs. Any differences observed were due to data coverage and ingestion timing rather than schema design flaws. This validation demonstrates that the Knowledge Graph is robust, extensible, and capable of supporting advanced analytical queries across independently developed datasets.

19 Changes After Meeting with Professor

After my first meeting with the professor, I received clear feedback about several issues in my project. He pointed out that scraping only the current roster was not enough and suggested that I should also scrape older rosters to improve the quality and completeness of the Knowledge Graph. He also mentioned that my weight-related data and nodes were not working correctly because of inconsistent formats. Based on this feedback, I decided to redesign my project structure. I converted all my CSV files into JSON format, including player rosters, coaches rosters, and the 2025 roster data. I also re-scraped the data to ensure that important attributes such as weight, position, and academic major were correctly extracted and stored. New relationships were created to better connect players with their teams, seasons, and majors.

In the second meeting, the professor asked me to run several queries related to player weight, academic major, and playing position. We discussed how these attributes can be combined to answer more meaningful questions using the Knowledge Graph. After reviewing the updated queries and the new data structure, the professor confirmed that the changes fixed the earlier issues and that the graph was now working as expected. He said that the project looks good and is ready for final submission. After this feedback, I finalized the Knowledge Graph, completed the report, and submitted the project. This process helped me understand the importance of improving a project based on feedback and building a more flexible and well-structured Knowledge Graph.